SOFTWARE-FIRST STRUCTURAL HEALTH MONITORING (SHM) PLATFORM FOR REAL TIME ANOMALY DETECTION

A PROJECT REPORT

Submitted by

DEVADARSHINI M [REGISTER NO: 211422243055]

DHIVYADHARSHINI S [REGISTER NO: 211422243067]

GOPIKASHREE P R [REGISTER NO: 211422243080]

in partial fulfillment of the requirements for the award of

the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

PANIMALAR ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

APRIL 2026

PANIMALAR ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report **"SOFTWARE-FIRST STRUCTURAL HEALTH MONITORING (SHM) PLATFORM FOR REAL TIME ANOMALY DETECTION"** is the bonafide work of **DEVADARSHINI M [211422243055], DHIVYADHARSHINI S [211422243067], GOPIKASHREE PR [211422243080]** who carried out the mini project work under my supervision.

SIGNATURE

Dr. S. CHAKARAVARTHI
SUPERVISOR

DEPARTMENT OF AI & DS,
PANIMALAR ENGINEERING COLLEGE,
POONAMALLEE,
CHENNAI-600 123.

SIGNATURE

Dr. S. MALATHI
HEAD OF THE DEPARTMENT

DEPARTMENT OF AI & DS,
PANIMALAR ENGINEERING COLLEGE,
POONAMALLEE,
CHENNAI-600 123.

Certified that the above mentioned students were examined in End Semester

Project Work (21AD1811) held on 10.04.2026

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION BY THE STUDENTS

We **DEVADARSHINI M [211422243055], DHIVYADHARSHINI S [211422243067], GOPIKASHREE PR [211422243080]** hereby declare that this project report titled "**SOFTWARE-FIRST STRUCTURAL HEALTH MONITORING (SHM) PLATFORM FOR REAL TIME ANOMALY DETECTION**" under the guidance of **Dr. S. CHAKARAVARTHI, M.E, Ph.D** is the original work done by us and we have not plagiarized or submitted to any other degree in any university by us.

ACKNOWLEDGEMENT

We would like to express our deep gratitude to our respected Secretary and Correspondent **Dr. P. CHINNADURAI, M.A., Ph.D.,** for his kind words and enthusiastic motivation, which inspired us a lot in completing this project.

We express our sincere thanks to our Directors **Tmt. C.VIJAYA RAJESWARI**, **Dr.C. SAKTHI KUMAR, M.E., Ph.D. and Dr. SARANYASREE SAKTHI KUMAR B.E., M.B.A., Ph.D.,** for providing us with the necessary facilities to undertake this project.

We also express our gratitude to our Principal **Dr. K. MANI, M.E., Ph.D.** and **Dr. S. PRASANNA DEVI, M.E, PGDHRM, Ph.D., PDF(IISC),** Chief Academic Officer who facilitated us in completing the project.

We thank the Head of the AI & DS Department **Dr. S. MALATHI, M.E., Ph.D.,** for the support extended throughout the project.

We would like to thank our supervisor **Dr. S. CHAKARAVARTHI , M.E, Ph.D, Professor,** Coordinators **Dr. K. JAYASHRI, M.E., Ph.D, Professor & Dr. S. RENUGA, M.E, Ph.D, Associate Professor** and all the faculty members of the Department of AI & DS for their advice and encouragement for the successful completion of the mini project.

DEVADARSHINI M

DHIVYADHARSHINI S

GOPIKASHREE PR

ABSTRACT

The problem of infrastructure failures does not only feature on the headlines, but it costs lives and billions of dollars in recovery. Bridges are cracking, buildings are sinking, dams are getting stress fractures with the strain of the years. The thing is that the majority of those problems evolve slowly and remain unnoticed until it is late. Manual checking assists, but it is rarely used, is a manual process, and practically impossible to theorize all.

We have developed a software based Structural Health Monitoring (SHM) system to address this issue in a different dimension which is described in this paper. We did not try to build a costly hardware or a complicated ML model: we initially concentrated on getting the software architecture correct. The system constantly executes simulated sensor values - vibration, strain, and temperature - and employs a two-layered system of anomaly identification: rules based on thresholds to flag anomalies in the short term, and machine learning with no supervision and using Isolation Forest and One-Class SVM models to detect deeper patterns in the long term.

Under the hood, it uses Flask backend with SQLite and SQLAlchemy to store data, which feeds a web dashboard that provides the user with live sensor data, the historical trends, and automated alerts to the user through email and SMS. The ML models are trained using the past sensor readings and re-trained automatically after 24 hours to keep up with the current readings. Initial tests using simulated streams were positive, of the two detection mechanisms, both performed well, alerts were generated as anticipated and the export and visualization options both withstood heavy load. The final outcome is a platform that is useful to put into practice, extend, and actually beneficial to engineers on the ground, not a proof-of-concept, sitting in the lab.

Keywords: Structural Health monitoring (SHM), IoT, Anomaly Detection, Isolation Forest, One-Class SVM, Machine Learning, Flask, Real-Time Dashboard, Smart Infrastructure.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	LIST OF TABLES	iv
	LIST OF FIGURES	vi
	LIST OF SYMBOLS, ABBREVIATIONS	vii
1.	**INTRODUCTION**	1
	1.1 Problem Definition	
	1.2 Objective	
	1.3 Divisions related to the project	
	1.4 Contributions of the work	
2.	**LITERATURE REVIEW**	8
3.	**SOFTWARE & HARDWARE USED**	14
4.	**PROPOSED SYSTEM DESIGN**	28
	4.1 Data flow diagram	
	4.2 Entity relationship diagram	
	4.3 Architecture diagram for proposed system	
	4.4 Entity Relationship Diagram	
5.	**PROPOSED SYSTEM IMPLEMENTATION**	38
	5.1 Module 1: Data Simulation and Ingestion	
	5.2 Module 2: Backend Processing & Anomaly Detection	
	5.3 Module 3: Dashboard Visualization	

6. **RESULTS & DISCUSSION** 45

6.1 System Performance Evaluation

6.2 Anomaly Detection Results

6.3 Visualization and Monitoring Output

6.4 System Responsiveness and Reliability

6.5 Discussion

7. **TESTING AND VALIDATION** 53

8. **CONCLUSION** 65

9. **FUTURE WORK** 67

REFERENCES 69

APPENDIX

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	Development Hardware Specifications	20
Table 3.2	ESP32 Specifications	21
Table 3.3	Arduino Uno with ESP8266 Specifications	22
Table 3.4	ADXL345 Accelerometer Specifications	23
Table 3.5	HX711 Strain Measurement Module Specifications	24
Table 3.6	DHT22 Temperature Sensor Specifications	24
Table 3.7	Communication Protocol Comparison	25
Table 3.8	Estimated Bill of Materials — Single Monitoring Node	26
Table 5.1	Module Summary	38
Table 6.1	Anomaly Detection Threshold Values	46
Table 7.1	Unit Test Cases — Sensor Simulation Module	53
Table 7.2	Unit Test Cases — Threshold Detection	54
Table 7.3	Unit Test Cases — ML Anomaly Detection	55
Table 7.4	Unit Test Cases — Alert System	56
Table 7.5	Integration Test Cases — Ingestion to Detection Pipeline	57
Table 7.6	Integration Test Cases — Database Operation	58

Table 7.7	Integration Test Cases — Dashboard to Backend	59
Table 7.8	System Test — Continuous Operation	59
Table 7.9	System Test — Anomaly Detection Scenarios	60
Table 7.10	System Test — ML Model Retraining	61
Table 7.11	Performance Test Results	62
Table 7.12	Validation Against Project Objectives	63

LIST OF FIGURES

FIGURE NO.	FIGURE DESCRIPTION	PAGE NO.
4.1	Flowchart of SHM System Operation	29
4.2	Overall Workflow of the SHM System	32
4.3	Layered Architecture of the Proposed SHM System	35
4.4	Entity Relationship Diagram	37
6.1	Real-Time Structural Monitoring Dashboard	48
6.2	Real-Time Sensor Trend Visualization	48
6.3	Historical Sensor Readings and Alert Classification	49

LIST OF SYMBOLS, ABBREVIATIONS

SERIAL NO.	ABBREVIATION	FULL FORM
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	AWS	Amazon Web Services
4	CPU	Central Processing Unit
5	CSV	Comma Separated Values
6	FK	Foreign Key
7	GCP	Google Cloud Platform
8	GIS	Geographic Information System
9	GPS	Global Positioning System
10	HCI	Human Computer Interaction
11	HTTP	Hypertext Transfer Protocol
12	IDE	Integrated Development Environment
13	IF	Isolation Forest
14	IoT	Internet of Things
15	JSON	JavaScript Object Notation
16	JWT	JSON Web Token
17	LSTM	Long Short-Term Memory
18	ML	Machine Learning
19	MQTT	Message Queuing Telemetry Transport
20	ORM	Object Relational Mapper

21	OS	Operating System
22	PK	Primary Key
23	RAM	Random Access Memory
24	RBF	Radial Basis Function
25	REST	Representational State Transfer
26	SHM	Structural Health Monitoring
27	SMS	Short Message Service
28	SQL	Structured Query Language
29	SQLite	Self-Contained Lightweight Database Engine
30	SVM	Support Vector Machine
31	UI	User Interface
32	URL	Uniform Resource Locator
33	UTC	Coordinated Universal Time
34	VS Code	Visual Studio Code
35	WSN	Wireless Sensor Network

CHAPTER 1

INTRODUCTION

1.1 Problem Definition

Every viewer of a television news report on the collapse of a bridge or the failure of a building understands the pattern, that the building or structure had been under stress months or even years before the tragic event, though no one was able to notice it in time. The issue which this project is attempting to address is the disconnect between the warning signs and the actual warning.

The infrastructure that we rely upon on a daily basis such as bridges, dams, buildings, industrial facilities was never constructed to be permanent. The materials wear out, the climate wears them down, and the loads are so great that they overload the structures which were not initially intended to carry them. All this does not occur in one day. The process of structural deterioration is virtually always sluggish, both a curse and an opportunity. A curse since it is easy to overlook. An advantage since when one is watching all the time you have the opportunity to catch it in time.

Periodic and manual structural inspection is the conventional one. Inspectors come to a construction periodically, examine what is accessible, compose a report and go. The structure is virtually unmonitored until the visit next time. This is quite satisfactory in the case of those structures that degrade gradually and in a predictable manner. It performs dismally on sudden load bursts, any latent interior damage and the type of cumulative stress build up that is not manifested through apparent physical evidence until it is too late.

SHM systems which use automated sensors are available and they solve some of these issues - but have their obstacles. The necessary hardware is costly. Installation is disruptive. Calibration is challenging technologically. Constant maintenance is an added expense. This may be done in case of big government-funded projects. In the context of smaller municipalities, in developing regions, or organizations with limited resources, it renders the continuity of monitoring unaffordable.

Another issue that has been minimally discussed with a lot of current systems is that they take data yet do not use it. A sensor reading storage system which does not analyze the sensor reads in real-time, visualize the results, or provide an alarm to an engineer does not contribute to faster decision making by the engineer. That data is stored there until it is manually read by a person, and this is where we are back to the exact same basic issue with periodic inspection with more data.

What is required is a monitoring platform that is cheap enough to be deployable with no significant equipment investment, intelligent enough that it will automatically raise an alarm when there is something wrong and useful enough that engineers and safety managers can actually act upon it. It is the gap that this project fills with a software-based SHM platform with real-time anomaly detection, live visualization and early warning alerts that is grounded to provide a practical modular and deployable solution not hindered by barriers limiting solutions currently in the market.

1.2 Objective

The objective of this project is quite simple to define, although the process of its implementation can consist of many moving components: create a structural health monitoring platform, which in practice, rather than on paper, is really going to work.

To be more precise, the following are the objectives:

To create a software-based SHM system which can be used to monitor structural conditions in real time without the deployment of physical sensor hardware at this point in time. The emphasis is to first have the software architecture correct, clean, and with the ability to interface with the real sensors when the moment arrives. The majority of SHM studies immediately move on to hardware or algorithms. We deliberately didn't.

To model realistic sensor behaviour of vibration, strain and temperature behavior that would be representative of how a real monitored structure would act like, both in normal operating cases, slow drift, and sudden anomalous spikes. It is

not just noise in the machine, the purpose of the simulation is to stretch the anomaly detection logic.

To deploy a dual-layer system of anomaly detectors, which entails both rule-level threshold checking and machine learning without oversight. The threshold system provides real-time, transparent classification as to what is considered to be Normal, Warning and Critical. The isolation forest and the one-class SVM model provide a statistical layer which captures patterns the thresholds may fail to see - especially slow, gradual deviations which the threshold does not cross per sec, but which are statistically abnormal.

In order to create a Flask based backend to process data-ingestion via REST API, preprocessing and anomaly-detection, database management and coordination of the alert system, all under a modular service architecture where each facet may be modified or swapped, we need to create a Flask-based backend.

To create a web-based dashboard that displays live sensor data, anomaly alerts and trending in a manner that is really valuable to the engineers and other safety managers, not merely technically correct, but actually readable and usable.

To introduce an automated warning system that will alert the appropriate officers by email and SMS in case of a Warning or Critical situation, but will include alert fatigue measures to prevent notification flooding in the case of long-term anomaly.

Storing and manipulating historical sensor data within SQLite and SQLAlchemy ORM in a form that facilitates trend analysis, performance analysis, and retraining the machine learning model in the future - the database should be easy to migrate to PostgreSQL or MySQL, once the system is scaled up.

In order to make the platform IoT-ready - the REST API ingestion interface is made in a way that physical sensors (via Arduino or ESP32 microcontroller) can begin to send data to the same endpoints with no changes to the backend. Migration between simulation and real hardware should not be a recreation, but more like a continuation.

Collectively, these goals characterize a system that provides the gap between the hypothetical research on SHM and something an engineer might deploy and trust.

1.3 Divisions Related to the Project

The project is at the crossroads between a variety of different disciplines and it would be a mistake not to admit that, without a single one, we could not have created the work we created. This is the contribution by each area.

The problem originates with **Structural and Civil Engineering**. Knowing why bridges crack, how material fatigue builds up, the number of load cycles taken over years, and what conditions in the environment make structures decay faster is civil engineering knowledge. It was what informed us about the parameters that really matter to act upon in monitoring: that from sudden spikes you actually know it is impacting or resonating; that with gradual increase you know it is deforming and becoming overloaded; and that with thermal expansion you know it is building up stress over time. Having no knowledge of this domain, we would be gathering data with no idea of what we were gathering.

The project is being developed according to the direction of **Internet of Things (IoT)**. The simulated data are employed in the current system, and all the design choices were taken considering the actual application of the IoT. The REST API ingestion layer is specifically due to the fact that microcontrollers such as Arduino and ESP32 use the HTTP protocol. The format of the payloads can be explained by the fact that the JSON format is what the IoT devices naturally give. The modular structure implies that physical sensors can be plugged without any change in the backend. IoT is not an issue of the future in this case but it was highly considered in the design of the whole system.

One of the first reasons that caused me to take this project is probably the fact that **Software Engineering** is the least credited division in SHM research. The systems can be deployed in practice but not just shown to work in a lab due to good software architecture: separation of concerns, modular services, clean APIs, good use of ORM, automated testing. The structure of Flask Blueprint, the three-layer

architecture, or the database design are all portrayals of the principles of software engineering that are implemented with purpose and not as an aside.

The intelligence layer is dealt with by **Machine Learning and Data Analytics**. The threshold detection is done using rules with immediate classification that is transparent. Structures do not necessarily collapse suddenly, however: occasionally corrosion is a gradual statistical drift that is never reached by any individual threshold. Isolation Forest and One-Class SVM are found there. They both represent unsupervised algorithms, which is important since real SHM situations practically never contain labeled examples of structural failures. The ML layer gets to know the appearance of normalcy and identify anomalies - even minor ones that rules cannot detect.

The concept of **Data Visualization and Human-Computer Interaction** is what makes the system actually useful as opposed to technically correct. A monitoring platform that retains the data in a database without making it presentable is not a complete solution. The dashboard was targeted at the final user - and engineer or safety manager who must be able to get a quick overview of what is going on, whether a spike is a real anomaly or a normal deviation, and whether to take action. HCI decisions include those of real-time graphs, color-coded severity indicators, historical filtering, and CSV export, as well as technical ones.

These five areas do not work separately in this project, they are actually integrated. The knowledge in civil engineering dictates what is to be monitored. IoT influences the input of data into the system. It depends on software engineering on how it gets processed. ML defines the way of identifying anomalies. Visualization defines the way in which results are delivered to the needy people. Take out any of these and the system lacks strength.

1.4 Contributions of the Work

The bulk of SHM research contributions can be considered in two groups - a new sensor design or a new algorithm. The contribution of this project is different: it is a software architecture, which makes the other two workable in reality.

This is what in particular this work offers:

An open-source hardware-free software-based SHM. The platform mimics actual vibration, strain, and temperature sensors, and then executes the whole monitoring process end to end; it reads the data, normalizes it, identifies abnormal patterns, stores the findings, displays them on a dashboard, and even sends alerts; it does all this entirely in software. This isn't just a demo. It is a proven platform on which real IoT sensors can be attached as and when they are ready, without the need to re-architecture the entire system. This contribution is making it true that it is possible to develop/create a complete SHM platform and test it without spending a rupee on hardware.

Two-layer system of anomaly detection. The system is not based on either of the rules or ML; it is a combination of the two. Rule-based threshold checking offers immediate and transparent classification into normal, warning and critical - simple to explain, simple to tune and simple to trust. The Isolation Forest and One-Class SVM models provide a statistical overlay that will pick up slow deviations and anomalous patterns which fixed thresholds fail to see. They both are unsupervised and that is the correct decision to make SHM since nearly never are the failure data of the infrastructure labeled in real-life settings. The combination has a wider coverage than either of the approaches.

Automated model retraining. The ML models do not train and remain constant. The system verifies age after every 24 hours and re-trains automatically after the last 7 days of data. This implies the anomaly detection is kept up to date since structural behavior changes with time - which all research prototypes utterly neglect.

An effective warning system that has fatigue protection. Anomalies are signaled by email alerts (Flask-Mail) and SMS alerts (Twilio) when detected on the platform. More to the point, it provides alert fatigue protection - a feature that disables the repetitive alerts of the same level in case it receives more than five alerts within an hour. It is the type of pragmatic detail which distinguishes between a deployable system and a research prototype. All the attempts to create alerts are also recorded to the database with a view to making them accountable and reviewing.

An **effectively useful dashboard**. Live sensor graphs, anomaly indicators, and statistical summaries are displayed as live in the web interface which is updated every two seconds. Time window and anomaly category can be used to filter historical data, which can be exported as CSV. The engineers and safety managers who have to know what is happening at a glance rather than NI scientists who are more at ease reading database tables in their raw form designed the dashboard.

An **interactive, IoT enabled structure**. Three-layer design ingestion, processing and storage, client and alert, implies that each of its parts can be upgraded separately. The ingestion layer based on the REST API can accept the same format of JSON that is sent by default by Arduino and ESP32 modules. Migration of SQLite to PostgreSQL only needs one configuration change. The only required modifications to swap Isolation Forest with an LSTM model will be made at the ML service layer. Such modularity is not something that happened but a conscious input of the architecture.

Collectively, what this project will provide is a full-fleet workable SHM platform that is more proximate to a real world implementation than most of the published literature - and that the software layer should be given as much focus as the sensors and algorithms that it binds.

CHAPTER 2

LITERATURE SURVEY

2.1 Farrar, C.R. & Worden, K., "An Introduction to Structural Health Monitoring," Philosophical Transactions of the Royal Society A, 2007.

- **Methodology**:
  The paper by Farrar and Worden is most likely the authoritative reference in SHM and justifiably so since it provided an effective framework of how structural monitoring can be thought of as a holistic system as opposed to a sensor deployment issue. Their method separates SHM into four steps: sensing, data acquisition, feature extraction and damage detection. The essence of this is that the damage will alter the dynamic properties of a structure its natural frequencies, mode shapes and damping ratios, and this alteration can be viewed in vibration measurements and statistical pattern recognition. When you know what the vibration signature of a healthy structure is, then anything that is not of the type of signature will be your damage indicator.

- **Merits**:
  - Offers a clean organized structure that can be used in various types of structure and monitoring conditions.
  - Detection technique Vibration-based monitoring is effectively and trustworthy by establishing it as a well-established detection method.
  - The modern ML-based anomaly detection is directly applicable to the statistical pattern recognition approach.
  - Provides future researchers with a common vocabulary and practice to develop.

- **Demerits**:
  - The framework presupposes a wide scale sensor usage, that introduces considerable cost of installation and routine.
  - Even the detected anomalies need expert interpretation to convert it into maintenance actionable decision.
  - The environmental factors (temperature and humidity have a significant impact on vibration measurements) are not completely resolved by the framework.
  - Expanding into large infrastructure systems is still complicated and expensive.

2.2 Lynch, J.P. & Loh, K.J., "A Summary Review of Wireless Sensors and Sensor Networks for Structural Health Monitoring," Shock and Vibration Digest, 2006.

- **Methodology**:
  The review by Lynch and Loh deals with a practical constraint of early SHM systems the wiring. Installing dozens of cables into a central system through a giant bridge or a building is costly, disruptive and generates maintenance nightmares of its own. They assert that the solution is a distributed wireless sensor network which consists of nodes that have accelerators and strain sensors scattered throughout the structure and wirelessly linked to a central monitoring point. The local data collection and transmission to the central system is done by each node in real time, and this allows constant monitoring without any physical cable infrastructure. The strategy is especially applicable in large buildings where it would be too difficult to have their wiring.

- **Merits**:
  - Removes the complexity and the cost of installation of wired connections in structures.
  - Allows monitoring of physically challenging buildings to be equipped with wired sensors.
  - The distributed architecture is self-scaling - adding nodes increases coverage without any change of architecture.
  - The wireless communication enhances the flexibility of reconfiguring the monitoring system when it is necessary.

- **Demerits**:
  - Battery life is operational factual challenge - and wireless nodes require a battery replacement or a recharge after some time, that increases maintenance load.
  - The structure or the surrounding environment can cause signal interference that will disrupt transmission of data.
  - Maintaining many wireless nodes in time is a challenging technical issue, and loss of data during transmission influences the reliability of monitoring.
  - Although the wiring expenses are removed, the total cost of deployment of large buildings is still considerable.

2.3 Li, H., Ou, J., & Zhou, Z., "Applications of Internet of Things in Structural Health Monitoring," Smart Structures and Systems, 2015.

- **Methodology**:
  Li et al. made the wireless sensor network idea a step further and put SHM in the IoT paradigm. Instead of merely a local wireless network of nodes, the framework links sensors which are accelerators, strain gauges, temperature sensors, etc., together using microcontrollers and the use of wireless protocols to send the sensor data to cloud-hosted storage, processing, and analysis services. The cloud performs the burden of computation, and sensors are used in data collection and transmission. The outcome is a system that can serve remote data monitoring everywhere, data processing, and a real-time alert generation in the case of anomalies detected. This is the architecture direction modern SHM systems are heading to, and is directly pertinent to the next stage of the project i.e. the IoT integration phase.

- **Merits**:
  - Real time surveillance without the need to have on the ground staff, when an object appears wrong it issues an alert.
  - Remote access implies that engineers will be able to look at several structures at a place.
  - The amount of data that continuous sensor incursion produces but does not store locally is managed by cloud storage.
  - The automation layer minimizes the need of manual reviewing.

- **Demerits**:
  - It is just that the system is effective as long as the internet connection is not failure-prone - a system failure equates to blind spheres of the monitoring process.
  - The outbound transmission of structural sensor data via networks brings cybersecurity issues that must be tackled in a real implementation.
  - Stable accuracies will need maintenance and sensor calibration, yet on-site work is still required on a periodical basis.

- Combining the cloud infrastructure with different sensor hardware introduces complexity to the system which might be hard to manage by smaller organizations.

2.4 Ye, X.W., Su, Y.H., & Han, J.P., "Structural Health Monitoring Based on Machine Learning Techniques: A Survey," Structural Health Monitoring Journal, 2019.

- **Methodology**:

  Ye et al. provided an overview of the use of machine learning in detection of structural damage, both supervised (such as Support Vector Machines and Artificial Neural Networks) and unsupervised (such as clustering) systems. The essence is that the ML models have the ability to learn the normal structural behavior based on sensor measurements such as vibrations, strain, patterns of displacement and sense anomalies which are indicators of damage. This is attractive as it is not confined to predetermined limits and can in theory detect complex patterns of damage which rule-driven systems cannot. The survey includes feature extraction, pattern recognition and damage classification of a variety of structural monitoring scenarios.

- **Merits**:
  - The ML-based detection can be more accurate than fixed thresholds, especially with gradual or complex damage patterns.
  - Computerized processing of large volumes of sensor data which would otherwise be cumbersome to analyze manually.
  - Is able to identify the patterns of concealed damage that cannot be identified by direct inspection and just simple thresholding.
  - It is possible to have models changing and getting better with time as we have more data thus our system retrains automatically after every 24 hours.

- **Demerits**:
  - The unsupervised methods are more realistic since the big and quality labeled data which the supervised models need are practically not available in the real world because of the structural failures.
  - The problem of model generalization is also factual, a model that has been trained on a bridge is not certain to perform in any way on a structurally similar bridge.
  - Not every infrastructure organization has the resources (computational resources and ML experience) to train and hyperparameter optimize.

- This is because the interpretation of complex models like deep neural networks is not easy and thus, is a problem of trust when the engineers are expected to make decisions based on an alert.

2.5 Sun, H., Burton, H.V., & Huang, H., "Smart Infrastructure Monitoring Using Cloud-Based Structural Health Monitoring Systems," Journal of Infrastructure Systems, 2020.

- **Methodology**:
  The author Sun et al. presented a cloud-based SHM system that combines sensor networks, wireless communication, and cloud computing into one monitoring system. Sensor information Vibration, strain, displacement Sensor information is transmitted wirelessly to cloud servers where it is processed, stored and analyzed. It provides real-time web dashboards to visualize the system and generates automatic alerts in case the structural behavior is more or less than the set limits. The cloud structure enables observation of different structures within a single location as well as remote accessibility of the engineers in any of the locations.

  This work is the closest in spirit, but has one major difference, we deliberately made the system local and software-based but not cloud-dependent. The responses are practical: the Cloud dependency leads to the connectivity requirement and operation cost that makes the implementation more challenging in smaller firms. Our architecture was developed by adding cloud integration as an extra step rather than an obligatory one.

- **Merits**:
  - The centralized nature of cloud storage makes it easy to monitor various structures in the same platform due to its centralization.
  - Live dashboards improve situational awareness and quicker decision making.
  - Remote access implies that one does not have to be on the premises of the structure to be able to monitor it.
  - The automatic alerts will enable a quick reaction without necessarily monitoring it.

- **Demerits**:
  - The internet connection is a hard addiction - any unavailability causes loopholes in monitoring which can be crucial.
  - It raises legitimate data security and privacy concerns to save the structural sensor data in the cloud.

- All infrastructure operators do not have the technical skills required in deployment and maintenance.
- Continuous transmission of sensor data to the cloud on a high-frequency basis creates huge volumes of data and it may prove to be costly to process.

CHAPTER 3

SOFTWARE & HARDWARE USED

SOFTWARE REQUIREMENTS

1. Operating System

- We did the project on **Windows 10/11**, which proved to be the most viable one in day-to-day developmental work. It can run Python and Flask development server without any configuration problems, and it is easy to test the web dashboard in a browser. In the development of this project, Windows is the easiest route to take by anyone who is replicating this project.

- **Ubuntu 20.04+** would be preferable in case of real deployment. It is lighter, more compatible with running the server processes with the constant load, and the support of the open-source tooling is much better. Linux tends to be more predictable than Windows in terms of flask applications in production.

- **macOS** technically, there is no issue using macOS with Python and Flask working just fine, but it has the effect that certain dependencies may require additional configuration before they can work, which is not the case with Windows and Ubuntu. It is practical but it is not our first suggestion.

- **Advice:** Windows 10/11 to develop and test. Ubuntu for deployment.

2. Programming Language

Python 3.10+ the entire backend is based on Python 3.10 and above. The choice of Python was not due to the fact that there is no other option but because it is indeed the right tool in this situation the ease of readability, its rich library support in data processing and machine learning, and the fact that it is a natural fit with Flask make it difficult to consider any other choice than Python in this type of project.

Core libraries used:

- **Flask** — manages the routing of all the backends, REST API endpoints and process requests.
- **NumPy** — Numerical processing of sensor data and feature preparation in machine learning.
- **Pandas** — operations of data processing and handling.
- **Scikit-learn** —Isolation Forest and One-Class SVM to detect anomalies with the help of ML, as well as StandardScaler to normalize features.
- **SQLAlchemy** — SQL database interactions.
- **Chart.js / JavaScript** — JavaScript and Chart.js are used as a live time dashboard and front-end charts.
- **Requests / JSON** — The data structure and an application programming interface communication.
- **Flask-Mail** — email notification delivery.
- **Twilio** — Critical alert delivery in case of anomaly in SMS.

There is one comment on the scikit-learn page, which bears mentioning in our system: it is not a choice. The initial requirements described it as optional in further ML integration but Isolation Forest and One-Class SVM are already available and operational. One of them is scikit-learn.

Note: Python 3.10 percent will be installed, as this is the best possible result and the most supportive version. Install the requirements on tips.

3. Frameworks and Libraries

- The most important backend framework is referred to as **Flask**. The examples of our usage include routing of REST API, receiving sensor data, ML prediction, dispatching alerts, and delivering the dashboard templates. The application can be subdivided into separate modules, such as main routes, API routes, and history routes, and each of them can be independently maintained, since Flask has Blueprint architecture.

- **SQLAlchemy ORM** All the database operations are dealt with by SQLAlchemy ORM. Representing Python objects as raw SQL would keep the code maintainable, and in the future, when we need to migrate the database, e.g. to PostgreSQL, a one-line changing of the configuration would suffice to convert the code instead of writing it anew.

- **Chart.js** The real-time line graphs of vibration, strain, and temperature are made on the dashboard using Chart. js. It reloads after every two seconds, when sensor data are supplied. We selected it instead of heavy visualization libraries due to its light weight and browser native nature and its particular emphasis on live data situations.

- **HTML, CSS, JavaScript** The dashboard frontend is created with HTML, CSS, JavaScript, and Bootstrap as the responsive layout framework and styling compatibility at various screen sizes.

- **Recommendation:** It is suggested to visualize with the help of Chart.js and use Flask as the backend. This combination is not heavy, well documented and adequate to meet the present day needs of monitoring.

4. Development Environment

- **Visual Studio Code** We used Visual Studio Code as a major editor during the project. Syntactic highlighting, Linting and debugging are well managed by the Python extension. The built-in terminal was easy to use and check logs without having to switch windows to run Flask.

- **Jupyter Notebook** Jupyter notebook can be used to test sensor simulation logic as well as data distribution exploration in advance before moving code into the main application. We applied it at the initial stages of simulation to ensure that our simulated sensor values had realistic distributions.

- **Anaconda** Anaconda eases the Python environment management, provided that you were operating on a machine with several Python projects; it ensures that there are no conflicts in dependencies between projects. Not absolutely required in case this is a standalone installation.

- **Proposal:** VS Main development code. Simulation logic Data exploration Jupyter Notebook Testing simulation logic in isolation.

5. Database

- **SQLite** In this, the major database is SQLite. It records all sensor values with their time, vibration, strain, temperature, ML anomaly flag, anomaly score and alert level. It also has the MLModel table where one can keep the metadata of trained models and the AlertLog table where all the attempts to get an alert can be found. SQLite is also suitable to our single-structure deployment, where we do not need a distinct server process, thousands of records in it are well-managed, and the SQLAlchemy ORM is designed to hide all database-specific syntax. There are three indexes on timestamp, alertlevel, and is anomaly that enable queries to be rapid as the information is added.

- **MySQL / PostgreSQL** The MySQL / PostgreSQL are important in cases when the system has to track numerous structures at the same time or when the data sizes exceed the comfort of managing SQLite. The SQLAlchemy ORM implies that one has to perform a change in the connection string within the config.py.

- **Recommendation:** Suggestion SQLite to develop and deploy one structure only. Production scaling with postgreSQL.

6. Browser & Client-Side

- **Google Chrome** The main testing browser during the development was Google Chrome. During development of the dashboard, the developer tools, especially the Network tab, which shows API calls, and the JavaScript debugging tool of the Console, came in handy.
- **Firefox** is also compatible with it and a good secondary browser.
- The front end (dashboard) is developed in **HTML, CSS, JavaScript, and Bootstrap**. It is responsive in nature, i.e. it scales on various screen sizes and does not break the layout. No frontend framework was utilized (React or Vue), basic JavaScript, and Chart.js were enough to meet the real-time update needs.
- **Advice:**Chrome to develop and test. The dashboard is also browser-compatible and can be used on any current browser.

7. Version Control & Collaboration

- **Git and GitHub** were used in version control during the project. To ascertain what has changed between the working versions, and roll back to a working state in case of a break, it was pretty simple to maintain a proper commit history. It can also be used as a permanent backup and it can be used to share the project.
- **Google Drive / OneDrive** The project reports, documentation drafts, and presentation files and anything that is not code were stored on Google Drive / OneDrive.
- **Recommendation:** Use informative commit notes to have all your code currently in GitHub. Access documents and reports using drive.

8. Visualization and Reporting Tools

- **Chart.js** All the in-dashboard visualization is done by Chart.js, both the real-time line graphs which update every two seconds displaying live vibration, strain and temperature measurements.

- **Microsoft Excel** can also be employed to review the data that was exported as CSV in the historical records page. The exports option in the dashboard creates clean CSV files that can be loaded directly to Excel as per trend analysis offline.

- **MS Word** Project documentation and report preparation were done using MS Word.

- **Recommendation:** The proposal served as the actual dashboard of the Chart.js. Excel to analyze the historical records of exports of data offline.

9. Optional Future Integrations

- These are not in place but the architecture was made with them in mind:

- **ESP32 / Arduino** — physical microcontrollers, which would be used instead of the Python simulation scripts. When this integration occurs, they would send the same payloads (JSON) to the same REST API endpoints, hence no changes are required at the back end.

- **MQTT** — a simple publish subscribe protocol that can be more efficient than HTTP in high-frequency sensor streaming of IoT. The system is currently ingested through HTTP; another ingestion path that can be added to the system is MQTT.

- **Firebase / AWS** — Remote monitoring, multi-structure deployment, large-scale data storage when the system grows too large to be localized on SQLite.

- **Twilio / SMTP** — this is already partially applied in the present alert service. It needs to be fully activated by placing credentials under the environment variables.

- **Recommendation:** The existing architecture allows the inclusion of all of these without the need to redesign the fundamental system.

HARDWARE REQUIREMENTS

10.1 Development Hardware

The hardware setup used in the development of SHM platform and its processing consisted of the following: As the project is software-first, there was no need to install any special structural monitoring devices at this phase. The entire monitoring pipeline could be run using standard computing devices.

Table 3.1: Development Hardware Specifications

Component	Minimum Requirement	Used During Development
Processor	Intel Core i3 or equivalent, 2.0 GHz	Intel Core i5, 2.4 GHz
RAM	4 GB	8 GB DDR4
Storage	10 GB free disk space	256 GB SSD
Operating System	Windows 10 / Ubuntu 20.04	Windows 10/11
Network	Local network or localhost	Localhost (127.0.0.1)
Display	1366 x 768 resolution minimum	1920 x 1080 Full HD
Browser	Any modern browser (Chrome/Firefox)	Google Chrome

It is also lightweight in hardware requirements. The SQLite database, Flask development server and Scikit-learn ML models all run freely within these specifications. The Isolation Forest and One-Class SVM models did not require any GPU at data volumes that were tested.

10.2 Future IoT Hardware — Planned Integration

Although the current system is being built on Python-based simulation scripts to produce sensor data, the architecture has been built to interface with physical IoT

hardware. The ingestion layer of the REST API takes in the same type of JSON that the next layer of hardware will generate by default, i.e., no changes to the back-end will be necessary when the actual sensors are attached.

The next phase deployment is seen to have the following hardware components.

Microcontroller — ESP32

The microcontroller used is recommended as ESP32, which is to be used when connecting hardware sensors to the SHM platform. It also has built-in Wi-Fi and Bluetooth, operates on 3.3 V, and has the ability to make HTTP POST requests directly to the Flask REST API endpoint without requiring any other networking hardware.

Table 3.2: ESP32 Specifications

Parameter	**Specification**
Processor	Dual-core Xtensa LX6, up to 240 MHz
RAM	520 KB SRAM
Flash Storage	4 MB (typical)
Wireless	Wi-Fi 802.11 b/g/n, Bluetooth 4.2
GPIO Pins	34 programmable GPIO pins
ADC	12-bit ADC, up to 18 channels
Communication Protocol	SPI, I2C, UART, HTTP, MQTT
Operating Voltage	3.3V
Power Consumption	80 mA active, 10 uA deep sleep
Cost (approximate)	Rs. 350 – 600 per unit

The ESP32 has a complete replacement of the Python simulation script. When linked to the sensor hardware below and with the IP address of the Flask server it opens to, it transmits the same JSON payloads, i.e. with vibration, strain, temperature, and time stamp, to the same /sensor-data endpoint. The backend receives and processes them just like simulated data.

Alternative Microcontroller — Arduino Uno with ESP8266

In cases where cost is a major factor of consideration, the Arduino Uno and an ESP8266 Wi-Fi module are a cheaper option. The read of sensors and formatting of data is done by the Arduino whereas the transmission is done wirelessly through the ESP8266.

Table 3.3: Arduino Uno with ESP8266 Specifications

Parameter	Specification
Processor	ATmega328P, 16 MHz
RAM	2 KB SRAM
Flash Storage	32 KB
Wireless (ESP8266 module	Wi-Fi 802.11 b/g/n
ADC	10-bit ADC, 6 channels
Operating Voltage	5V
Communication	SPI, I2C, UART
Cost (approximate)	Rs. 250 – 450 per unit (Arduino + ESP8266 combined)

Vibration Sensor — ADXL345 Accelerometer

ADXL345 is three-axis digital accelerator which can be used to measure structural vibration. It talks via either I2C or SPI to either the ESP32 or Arduino and reports the

vibration readings in g-force units that can be transformed to fit the range of vibration parameters available on the platform.

Table 3.4: ADXL345 Accelerometer Specifications

Parameter	Specification
Measurement Range	±2g, ±4g, ±8g, ±16g (selectable
Resolution	13-bit at ±16g range
Output Interface	I2C / SPI
Operating Voltage	2.0V – 3.6V
Sampling Rate	Up to 3200 Hz
Sensitivity	256 LSB/g at ±2g range
Operating Temperature	-40°C to +85°C
Cost (approximate)	Rs. 150 – 300 per unit

The ADXL345 is especially well suited to bridge and building vibration monitoring as the high sampling rate and variable scan limit can enable the device to record structural sway and impact events at extremely low and extremely high frequencies, respectively.

Strain Sensor — HX711 with Strain Gauge Load Cell

A strain gauge load cell is attached to an HX711 24-bit analog-to-digital converter module in order to measure structural strain. The HX711 amplifies and digitizes the very tiny resistance variable of the strain gauge and feeds out the output to the microcontroller across a simple two-wire connection.

Table 3.5: HX711 Strain Measurement Module Specifications

Parameter	Specification

ADC Resolution	24-bit
Input Sensitivity	±40 mV full scale
Gain	128 (Channel A) or 64/32 (Channel B)
Data Rate	10 SPS or 80 SPS (selectable)
Operating Voltage	2.6V – 5.5V
Interface	Two-wire (Clock + Data)
Operating Temperature	-40°C to +85°C
Cost (approximate)	Rs. 100 – 200 per unit

The strain gauge itself is directly bonded to the monitored element of the structure, the beam, column or slab surface. Resistance variation due to deformation of the material is recorded by the HX711 and sent as strain values to the monitoring pipeline.

Temperature Sensor — DHT22

The DHT22 is a digital temperature and humidity sensor which can supply true temperature values that can be used in the thermal needs of the SHM platform. It is cheap, user friendly, and does not need any extra ADC hardware.

Table 3.6: DHT22 Temperature Sensor Specifications

Parameter	Specification
Temperature Range	-40°C to +80°C
Temperature Accuracy	±0.5°C
Humidity Range	0% – 100% RH

Resolution	0.1°C
Sampling Rate	Once every 2 seconds (matches dashboard refresh rate)
Interface	Single-wire digital
Operating Voltage	3.3V – 5V
Cost (approximate)	Rs. 150 – 250 per unit

The 2-second sampling rate of the DHT22 is the same as the 2-second dashboard polling rate of the existing system and thus it fits the platform perfectly without having to make any timing modifications to the backend.

10.3 Communication Protocol Options

The existing system depends on the HTTP operation over the Wi-Fi as a means of data transfer, which operates with both ESP32 and Arduino/ESP8266 constructs properly. Another protocol supported by the architecture is the MQTT that can be used in cases when the frequency of sensors requires a higher frequency.

Table 3.7: Communication Protocol Comparison

Protocol	Current Support	Best Used When	Overhea
HTTP (REST API)	Fully implemented	Standard Wi-Fi deployments, low-to-medium frequency	Medium
MQTT	Planned — future integration	High-frequency sensors, bandwidth-constrained networks	Low
Bluetooth (BLE)	Not currently supported	Very short-range, low-power deployments	Very Low

Since all the IoT microcontrollers support it naturally and no extra infrastructure is needed in the form of a broker, HTTP was selected to be used in the current

implementation. In future versions, MQTT will be supported as an additional ingestion point, when the deployment needs a higher transmission rate or works on a limited wireless network.

10.4 Approximate Hardware Pricing of Deploying a single structure.

The estimated bill of materials to implement SHM platform on one monitoring point in a real structure with the use of the above hardware components is as follows.

Table 3.8: Estimated Bill of Materials — Single Monitoring Node

Component	Quantity	Unit Cost (approx.	Total Cost (approx.
ESP32 Microcontroller	1	Rs. 500	Rs. 500
ADXL345 Accelerometer	1	Rs. 250	Rs. 250
HX711 + Strain Gauge	1	Rs. 300	Rs. 300
DHT22 Temperature Senso	1	Rs. 200	Rs. 200
Connecting wires and PCB	1 set	Rs. 150	Rs. 150
Power supply unit	1	Rs. 350	Rs. 350
Weatherproof enclosure	1	Rs. 400	Rs. 400
Total per monitoring nod			**Rs. 2,150**

The backend server (Flask, SQLite and the dashboard) can be deployed on any computer or inexpensive server. A Raspberry Pi 4 (around Rs. 4,500) is all that is needed to serve the entire backend of a single structure deployment, which makes the overall cost of a complete single-node monitoring system about Rs. 6,650. This is a small portion of the commercial SHM hardware solutions which are usually in the lakhs of rupees to install.

10.5 Recommendation

To develop and test it will work with a standard laptop or desktop with Windows 10 or Ubuntu 20.04 and at least 4 GB RAM. There is no extra hardware needed.

To deploy a single structure production - one ESP32 node per monitoring point, with ADXL345, HX711/ strain gauge, and DHT22 sensors and connected to Wi-Fi to a Raspberry Pi 4 or any Linux server to run the Flask backend.

In the case of multi-structure deployment - same node hardware per structure, with the backend being migrated to a cloud server (AWS, GCP or Azure) that is running PostgreSQL rather than SQLite.

CHAPTER 4

PROPOSED SYSTEM DESIGN

The logic of our system formulated is quite simple, i.e., data has to go in a single direction, each stage, which is wildly different, has to have its specific job, and no one stage should have to know anything about the inner workings of the other. It streamlines this system to debug, extend it and reason about it when it goes wrong.

There are four issues that the general platform handles: creating or receiving sensor data, processing and validating sensor data, performing anomaly detection, and presenting results to users in real-time. The database, the alert system and the ML models are the four other main functions that are supported by the rest of everything.

4.1 Data Flow Diagram (DFD)

The Data Flow Diagram is a lineage of the process in which sensor reading goes through after it is generated until the time the sensor is shown on the dashboard or an alert is triggered. It is the flow that is the key to the comprehension of the system in general.

Stage 1 — Data Generation

The process starts with the simulation of the sensor. The sensor service generates Python scripts, which generate vibration, strain and temperature information that act more realistically (temperature varies with time of day, strain varies with temperature with a thermal expansion factor, temperature varies with strain with a thermal expansion factor). It is sometimes purposefully introduced, as spikes to the simulation so that there is something interesting to anomaly detection logic. In future deployments, this step is substituted by physical sensors to which accelerators and strain gauges, as well as thermistors, can be connected to ESP32 or Arduino microcontrollers. There is nothing that is different to the eyes of the rest of the system.

Stage 2 — Data Ingestion

The resulting readings are sent to Flask backend with the help of the HTTP POST request to sensor-data endpoint. Each request carries the payload of sensor value in a form of a JSON having a timestamp. The logic of selecting the REST API formatting is that all the microcontrollers employed in the IoT are built with the provision of HTTP devices - once the simulation is substituted with real hardware, it will transmit the identical JSON to the identical endpoint with no changes made in the back-end.

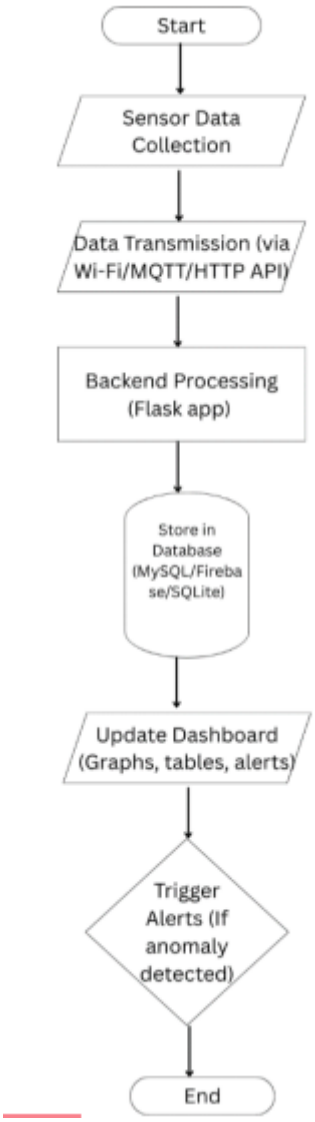

Fig 4.1: Flowchart of SHM System Operation

Stage 3 — Data Processing and Validation

A processing pipeline preprocesses the incoming data before any form of analysis is performed. The validation procedure will be used to make sure that it is considering

the right fields and getting rid of any unfinished fields. This is followed by the Standard Scaler normalization of all the three sensor measurements to a common range - otherwise, the temperature readings in the range of 20-45 C would overwhelm the ML model in the range of 0-1 due to the disparity in the scale. Finally, the timestamps are reformatted into the UTC format to make them inevitable to be stored and queried in a database in the same manner.

Stage 4 — Anomaly Detection

Two independent processings of readings are detected. Each sensor value will be compared with set limits which will be warning at more than 2.0, critical at more than 2.5; strain at more than 0.5, critical at more than 0.7; temperature at more than 35 C and critical at more than 40 C. Simultaneously, the ML service runs the Isolation Forest model on the reading, producing an anomaly score, which could be taken to mean a statistical abnormality of the reading, as compared to learned normal patterns. An alert level of threshold and the ML anomaly flag and the score are all appended to the reading and then pass onto the next step.

Stage 5 — Data Storage

The annotated and full processed reading is inserted into the SQLite database via SQLAlchemy ORM. Each record keeps record of the raw sensor values and time, ML anomaly flag, anomaly score, alert level and alert sent. It is this accruing history that makes trend analysis, retraining of the ML model and historical filtering possible on the dashboard. The queries are supported by three database indexes on timestamp, alert level and is-anomaly, so that the queries are fast as new records are added.

Stage 6 — Visualization and Alerting

The dashboard receives the latest reading after every two seconds and the live graphs on all three sensor parameters are overlaid with them. There are also color coded signs on the dashboard that indicate immediately in the event that the reading shows a Warning or Critical level of alertness. Meanwhile, the alert service will make sure that some outside notification will be provided - email in the case of a Warning or Critical event, SMS in the case of Critical event only - provided that the alert fatigue

protection will limit the number of notifications of the same level of importance to five in an hour.

The six stage flow ensures that reading is verified in advance of analysis, analysis is verified in advance of storage and storage is verified in advance of visualization. The system is actually modular and not just claims to be so because of no stage stepping forward or no stage needing the implementation details of another.

4.2 System Workflow

It is not a linear system, but rather a loop, and with the sensor reading every two seconds, processes it, compares it with abnormalities, stores it, and modifies the dashboard. It is this cycle which assists us in comprehending how everything is assembled.

That is actually what happens during each iteration:

The sensor service generates another vibration, strain and temperature value based on the above mentioned simulation patterns. The HTTP POST protocol immediately adds this message to the Flask database-server at sensor-data endpoint in the form of a JSON object. This is transmitted to the backend whereby it is run through the preprocessing pipeline, e.g. validation, normalization, standardization of timestamps and passed on to the ML service.

The trained Isolation Forest model attempts to forecast anomalies by the ML service. When there is no available trained model at the time, i.e. on the first start-up, when there is not enough data to use, it smoothly degenerates and just reports the reading to a threshold-based detector. Upon possession of the model, a score of anomaly together with the prediction of True/False is produced. The score, and even the flag is affixed to the reading.

Throughout the non-dependent mode of operation, the threshold checker is then run over the ML result. It will be comparing all sensor values with the established limits and will establish an alert level, which could be: Normal, Warning or Critical. This is regardless of what the ML model purported where both detection mechanisms are not

supposed to complement each other, but are supposed to be complementary to each other.

The raw value of the reading, ML anomaly flag, anomaly score and its alert level are put to the database. In the cases of Warning or Critical alert level, and in the event that the alert fatigue check is successful, the alert service is released. Email is activated upon warning events. The critical events trigger email and SMS. The success or failure of the effort to send an alert is counted in the AlertLog table.

Finally, the dashboard that is polling the /sensor-data endpoint every two seconds in the user browser receives the reply and displays the live graphs, the status indicators and anomaly highlights in real-time.

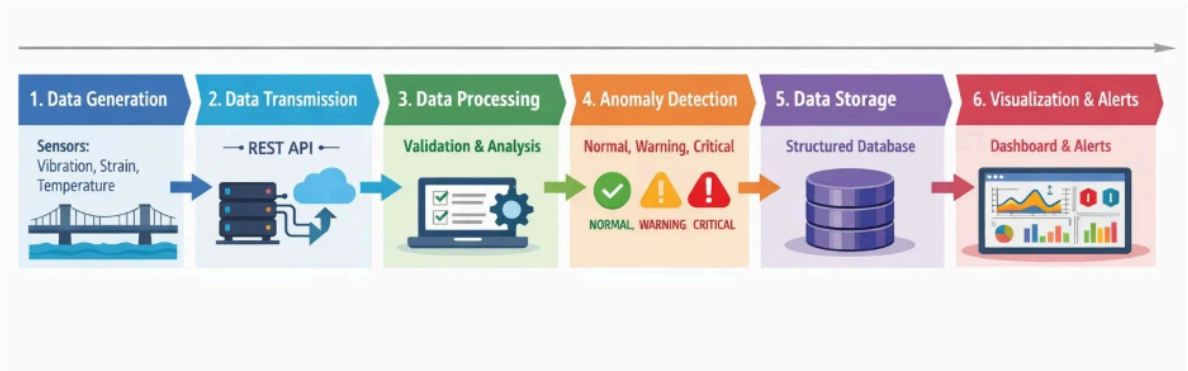

Fig 4.2: Overall Workflow of the Structural Health Monitoring System

The entire process would require seconds. This flow may be shown graphically in the form of Fig 4.2 where data are generated during the process of the transmission, processing, detection, storage and visualization and this is repeated.

4.3 Architecture Diagram for Proposed System

The system architecture has three layers. Each of the layers performs a specific task and communicates with the adjacent layers- that is, any of the layers may be upgraded or replaced without affecting the others. This is not simply a design principle that was declared to the paper but it is a decision that informed all the decisions on implementation.

Layer 1 — Sensor & Data Ingestion Layer

This is where the data enters into the system. That is currently Python simulation scripts generating sensor readings on a time basis. On future applications it suggests physical sensors accelerators to measure vibration, strain sensors to measure dislocation of the structure, thermistors to measure temperature over ESP32 or Arduino microcontrollers with Wi-Fi or MQTT.

The fact that the ingestion interface is not hardware-dependent is the most significant part of the design choice made on this layer. The information is sent in the form of the JSON over HTTP regardless of what code the origin would be (a Python script or ESP32 module). The other components of the system need not be aware of the distinction. This is what enables simulation-software transition to real-hardware transition to be configuration change and not system redesign.

The layer exposes the terminations of the REST API that takes sensor sensing measurements in a predefined format of a JSON structure consisting of vibration, strain, temperature, and time. Data can be sent to this layer using any device with an HTTP POST request - in effect, all of what the available IoT platforms offer.

Layer 2 — Processing & Storage Layer

This is what system is all about - the intelligence absolutely. Between this layer lies the Flask backend which accepts data into it through the ingestion layer and it structures three sub-components which are the preprocessing pipeline, the anomaly detection service and the database.

The incoming reading is initially processed on the preprocessing pipeline. It verifies that the data is complete, normalizes sensor values by use of StandardScaler to bring all the sensor values to the same scale, and puts the timestamps to the standard scale. At this stage, dirty data is eliminated before it can enter into the ML model or the database and corrupt it.

The next running service is the anomaly detection service. It is running on two tracks at a time. The threshold based track is used to verify sensor values with the set limits in config.py and generate an alert level. The ML track executes the Isolation Forest model, which is trained on the previous 7 days of past readings and which takes five

features, namely, vibration, strain, temperature, hour of the day, and day of week, and yields an anomaly score and a binary flag. The result of having the two tracks together is that the system captures both the sharp threshold violations as well as the slight statistical abnormalities.

The ML models are automatically retrained after every 24 hours. The retrain_if_needed approach makes age checks on every call and when the active model is past the retraining interval it runs a new training invocation - disabling the old model and re-training it to replace an old one with a new one. This maintains the up to date of detection because the pattern of structural behavior changes.

Everything is held in the database. SQLite engine with SQLAlchemy ORM includes three tables, i.e. SensorReading which stores sensor data and ML discoveries, MLModel that stores metadata of trained models, and AlertLog that stores all the attempts of alerts. The queries are also fast because three timestamp, alert level, and is anomaly indexes are used and therefore the queries are fast even when the records are in the thousands.

Layer 3 — Client & Alert Layer

This is the layer that becomes in direct contact with the users. It is made up of two components which are the web dashboard and the alert service and both are driven by what the processing layer provides.

The web dashboard is created based on Flask templates, JavaScript and Chart.js. It samples / sensor- data at a rate of one in two seconds and shows three live line graphs one of which is a graph of the vibration, strain and temperature. The irregularities are immediately indicated with the color coded warnings of the magnitude, yellow, Warning; red, Critical. The history page gives the views of the past readings filtered by time and anomaly type and has a CSV export option to allow offline analysis. The dashboard is directed at the engineers and the safety managers who would need to be presented with the information about the structural state in a brief manner, as opposed to data scientists who would feel comfortable reading the raw database output.

The alert service is not connected with the dashboard. A Warning or a Critical reading is made through and a check of a table AlertLog is made against the recent alerts of the same severity. It sends the notification, that is, email in case of a Warning and Critical event and SMS in the case of Critical event only, when it is not sent in the preceding hour within the number five or more, having adopted Flask-Mail and Twilio respectively as the sending service. All the dispatch attempts are documented in case of dispatch success or failure and hence full records are always available of what and when was sent to whom.

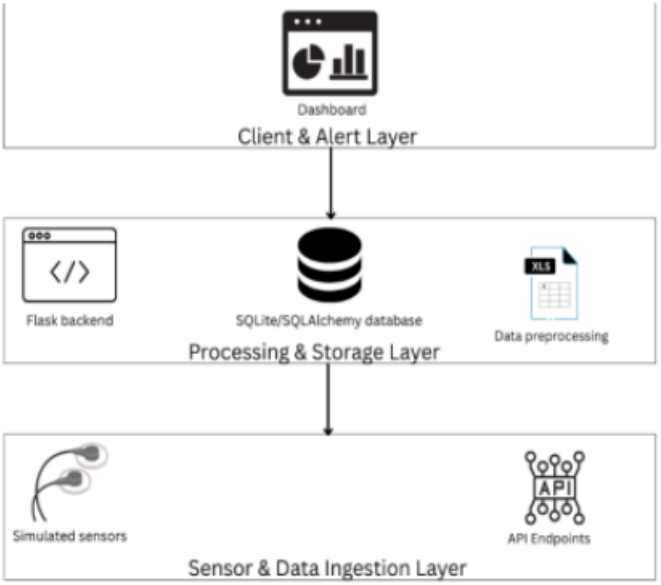

Fig 4.3: Layered Architecture of the Proposed SHM System

The three layers model suggests that the system can be literally extended in the most important directions. Just Layer 1 is contacted with the further addition of real IoT sensors. The only layer to cross over in changing the ML model Isolation Forest to LSTM is Layer 2. The reorganization of dashboard only goes to Layer 3. Each change to either of the layers incurs changes to the other - and this is the whole concept behind the architecture.

4.4 Entity Relationship Diagram (ERD)

The system supports a database, which is simply designed. It contains three tables and a knowledge of the relationship between the three tables would be enough in knowing how everything that the system requires to store and access is done.

The primary table is SensorReading. Every sensor response that drives into the system is driven into this whether in the Python simulation or later into a real physical sensor of the IoT. One record will hold an ID, UTC of the time of capture of the reading, the three sensor readings (vibration, strain, temperature), the ML anomaly flag (True/False), the continuous anomaly score of the Isolation Forest model, the alert level (Normal/Warning/Critical) and a boolean indicating whether or not an alert was reported during the time of capture of the reading. It is on this table, all things are built around--which is read on the dashboard, which is being trained by the models of the ML, which is being enquired of historically.

The second table is MLModel. Each time a system is training a new model of Isolation Forest or a One-Class SVM, a new record is made. It has the model ID, the name given to the algorithm employed, the time at which the training was performed, number of samples employed, contamination parameter, which was set and a flag indicating whether the model is in use or not. There is only a single model which can be active at any given time. When the retraining cycle of 24 hours is being done, the old model is being put in deactivation and the new model is being put in activation. It is this table that helps the system to maintain the history of the model and roll back when the need arises.

The third table is AlertLog. A record is logged whenever the alert service is successful or not in sending a notification. It has a special identifier of the alert (ID), the ID of the SensorReading that generated the alert, the type of alert (email or SMS), the receiver, the date and success of dispatch. This table is what the alert fatigue protection is guarding (before another notification is posted) and the system will refer to AlertLog and keep track of the number of times that the same level of alert has been sent within the last hour.

The relationship of these tables is simple. SensorReading and AlertLog are related - each record of the latter is an attachment to the former SensorReading that created it.

MLModel does not have the foreign key relationship with SensorReading but active model during the reading is the one that generated the anomaly flag and the score in that reading.

It was not a chance that the schema was so simple. There were clear, no unnecessary normalization and three tables. The table SensorReading is rapid to add - in this case every two seconds - and, therefore, it is significant to keep it lean and properly indexed as a key to the performance of queries. It has three indexes on: timestamp, alert level, and whether it is anomaly, which allow the queries on historical and trending queries to be fast in the situation when records are in the thousands.

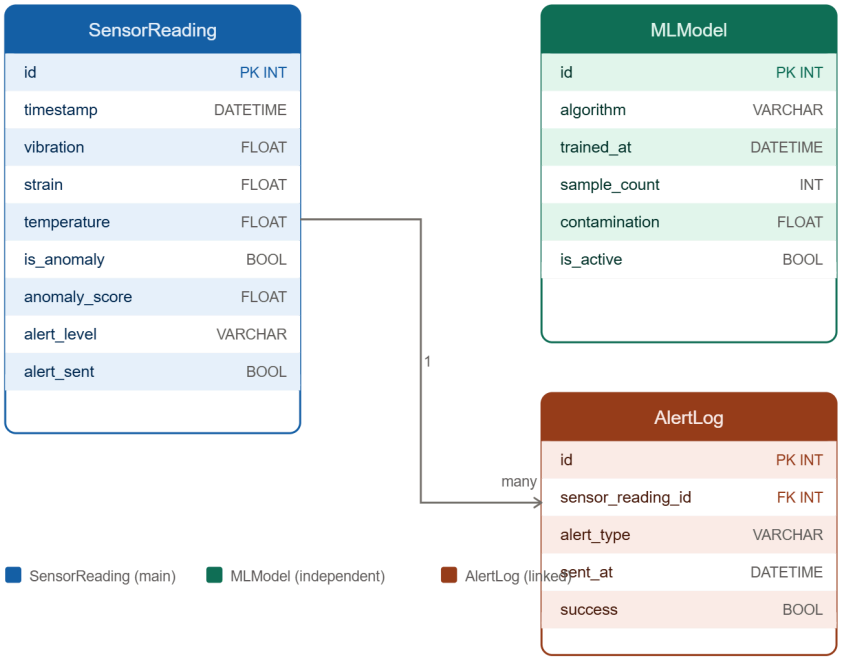

Fig 4.4 Entity Relationship Diagram of the SHM Database

CHAPTER 5

PROPOSED SYSTEM IMPLEMENTATION

Construction of the system was much concerned with the actualization of the architecture decisions of the design chapter. The modules needed to work together in a clean manner not merely appear nice on a diagram. What ensues is a candid description of the way each of the pieces was followed through and the decisions that were made in the process.

Table 5.1 — Module-wise Implementation Summary

Module	Function	Technology Used
Module 1	Sensor data simulation and ingestion	Python, REST API, JSON
Module 2	Anomaly detection and backend processing	Flask, Isolation Forest, SQLite
Module 3	Dashboard visualization and alerts	Chart.js, Flask-Mail, Twilio

5.1 Module 1: Data Simulation and Ingestion

Objective:

Because the project is aimed to validate the software architecture instead of testing concrete hardware, this module will be used instead of physical sensors, which will be substituted by Python-generated simulated data. The idea is to generate sensor readings which act sufficiently like reality to provide the anomaly detection logic with a realistic work out - not a consistent random noise, but values that vary, spike on occasion and exhibit behaviors in accordance with real structural sensor behavior.

Working Process:

In sensorservice.py, sensor simulation produces three parameters at a time. Temperature is a daily sinusoidal distribution with mean higher at midday and lower

in the evening with some random noise to represent environmental variation of +-5degC. To derive vibration based on temperature a thermal expansion factor is used, that is, larger temperature generates a slightly higher baseline vibration, and other random variation is increased +-1.5 units that were capped to at least 0.1. The strain is independently distributed with equal random values of 0.0 to 0.8 which denote independent random variations in load of the structure.

Sporadic spikes are introduced into the simulation. In their absence, the anomaly detection logic would never be triggered in testing which would make it impossible to check whether the system actually works or not. There is a purpose to the spikes, which are weighted, to make them look more real, like sudden events of loading, or the strain of weather.

Every produced reading is a JSON object which is forwarded by means of HTTP POST to the sensor-data endpoint of the Flask back end. The payload has the vibration value, strain value, temperature value and a UTC timestamp. This particular type of the JSON format has been selected as it is the default format sent when the ESP32 and Arduino modules are used over Wi-Fi, i.e. when actual hardware finally replaces the simulation, it transmits the same payloads to the same destination. The backend is simply unaware of the origin of the data (whether it was a Python script or a physical sensor).

Outcome:

The sensor readings are structured, realistic and constitute a continuous stream of data that is pumped into the monitoring pipeline every two seconds, which initiates the entire processing and detection cycle on each iteration.

5.2 Module 2: Backend Processing & Anomaly Detection

Objective:

This is the module where system intelligence resides. It takes raw sensor information, cleans and normalizes this information, processes it against two parallel anomaly detection systems and stores the results and issues alerts where necessary.

Data Processing:

All incoming reading is run through a preprocessing pipeline ahead of any analysis taking place. Testing the validation step validates missing or incorrectly-formed fields - any incomplete payload is rejected on the spot instead of it being allowed to corrupt the database or bias the ML model. Approved readings are then subjected to Standard Scaler normalization that normalizes all three sensor values to the same scale with zero mean and unit variance. This step is necessary since otherwise, the temperature values of the range 25-40degC would numerically dominate the ML model over the strain values of the range 0.0-0.8, merely because it is a larger number - biased anomaly detection. Lastly, the timestamps are changed to the UNIX coordinate to ensure that they are uniformly stored and queried.

Anomaly Detection — Threshold Layer:

The initial detection system is a rule based and open system. Config.py is used to set up thresholds that delimited each sensor type. A Vibration reading of 2.01 causes a Warning alert level and 2.50 causes Critical one. Strains of more than 0.5 result in Warning, and more than 0.7 result in Critical. At temperatures higher than 35degC, a warning is issued, at higher temperatures (indicating more seriousness) above 40degC, a critical is issued. These thresholds are not hardcoded but can be configured depending on the type of structure, and the range where that structure should be operating at.

This is not a limitation of the simplicity of this approach. The use of threshold-based detection is straightforward, simple to articulate to non-technical parties, and uncovers sharp and obvious anomalies promptly.

Anomaly Detection — Machine Learning Layer:

The training of the Isolation Forest model is also executed on the same reading by the second detection mechanism. Isolation Forest is a non-supervised algorithm - it gets to know how normal sensor patterns should be without necessarily having labeled examples of structural failures, which can hardly occur in the real infrastructure monitoring context. The model takes five input features, vibration,

strain, temperature, hour of day and the day of week. The time characteristics enable the model to be able to explain patterns of natural variation - a structure that vibrates more as a result of high traffic periods is a normal phenomenon rather than an anomaly.

The model gives two outputs to every reading, the binary flag of whether the reading is anomalous and the continuous anomaly score through the decision function approach. A negative score is a sign of an irregularity - the lower the score, the more abnormal the indicator. Scoring above zero shows normal results. Both outputs are logged with the sensor numbers which provide the dashboard with a greater image of what is being visualized as compared to the yes/no flag.

There is also an option of a One-Class SVM model with an RBF kernel. It puts a boundary to the normal data distribution and considers the readings beyond the normal data distribution as abnormal. The two models can be trained and switched using the API without system downtime.

The ML models are auto retrained.The retrain when required approach verifies the age of the model in use every time it is called. In case the model is more than 24 hours old and has at least 50 new readings, it will turn the old model off, then it will train a new model using the last 7 days of sensor history and save model metadata to the ML Model table and activate it. This makes the anomaly detection to be up to date because structural behavior changes with time, something that cannot be handled at all by a static model.

Database Storage:

Each reading containing its raw sensor reading, preprocessing output, ML anomaly flag, anomaly score and threshold based alert level - is inserted into the Sensor Reading table in SQLite using SQL Alchemy ORM. The ORM implies that the application interacts with Python Objects instead of direct SQL and therefore the code is easily readable and database migration to other databases is simply a one-line change in configuration. Three indexes of timestamp, a level of alert and an anomaly maintain the queries rapidly as the records grow.

The table of the Alert Log captures all the attempts to dispatch alerts - to whom, what was included and when the alert was actually dispatched, and was it effective. This will provide a full history of accountability of all the notifications.

Alert Dispatch:

Where the reading is accompanied by a Warning or Critical alert level, the alert service examines the Alert Log of recent alerts of the same level. It will send the notification in case less than five have been received during the past hour, which is the limit of alert fatigue. Where warning events occur, an email will be sent to the user through Flask-Mail that displays the sensor reading details and ML anomaly score in full. Critical events send both email and SMS through the twilio API. The SMS text is trimmed down to the character limitations but still contains the alert level, sensor values and date.

Outcome:

Any incoming reading is validated, normalized, classified using both threshold rules and ML prediction, and stored with full metadata and escalated using the alert system where the conditions are warranted. The dual-layer detection ensures that sharp threshold violations as well as the subtle statistical variations are captured by the system that would have been overlooked by rules.

5.3 Module 3: Dashboard Visualization

Objective:

The component of the system that is used by engineers and safety managers is the dashboard. This was meant to create something that is truly useful (not just technically functional), that is, information must be immediately interpretable without having to know what the underlying database schema or ML model outputs are.

Dashboard Architecture:

The front-end is developed using the Jinja2 templating engine of Flask, Bootstrap to support a responsive layout, vanilla JavaScript to support interactivity and Chart.js to support real time graph generation. No super backend framework was employed, the real-time update needs were fulfilled using a simple polling system, where the endpoint / sensor-data is called after every two seconds and the charts are updated in situ.

Real-Time Visualization:

This isolates Chart.js line charts showing real-time data of vibration, strain and temperature respectively. Every graph has a rolling window of the last 20 data values hence users are able to view the current trend and the chart is not overcrowded. The graphs are smoothly updated within the poll cycle by Chart.js using the no-animation update mode that avoids flickering of the visualization in case of high frequencies.

There are also three status cards above the graphs indicating the present numerical value of each of the parameters, color-coded according to the present value of the alert level. Default theme color presents normal readings. Warning readings turn amber. Critical readings turn red. The color coding is also extended to the card borders, and thus, the overall system status leaks out immediately prior to a user even examining the graphs.

Historical Analysis:

The history page at /history gives a table of all of the stored readings with their timestamps, sensor values, badges indicating anomaly status, badges indicating level of alertness, and ML anomaly scores. The user can filter by date or category of anomaly and can export any filtered data as a CSV file to do a custom analysis of it offline in Excel or any other tool. Filter and export features continued to be the most practical features to be used in testing - that is what you actually do when you want to explore an anomaly event after the fact.

Alert Indication:

In case the backend is giving a Warning or Critical reading, the dashboard will show an alert banner at the top of the page which is dismissible with the intentional violation of the specific threshold. Critical warning banners are designed in red and amber. Banners can be closed after 10 seconds automatically but can be closed manually. The status cards and the graph borders color coding are always updated at the same time, hence the alert is not only displayed on one part of the interface, which is a notification area.

Connection Status:

The bottom of the dashboard has a status indicator indicating that the system is receiving information either has an active polling status of Connected with a Last update: [timestamp] when the polling is successful, or Connection lost with Attempting to reconnect when it is not. This is a minor detail but a quite practical one, in the real-life situation in the monitoring process, an engineer should be aware right away whether the data feed is dead or not.

Outcome:

The dashboard offers a full real-time structural health monitoring interface including live sensor graphs, anomaly indicators, statistical summaries, historical filtering, and a CSV export created to be useful to users who must know the conditions of structurally relevant vehicles and take actions based on the information they see.

CHAPTER 6

RESULTS AND DISCUSSION

Working on the system with simulated data streams provided a good understanding of what the system can do at this point. The brief is that it works, it works constantly and dependably, but that there exist honest limitations that are worth admitting as well as the outcomes. This chapter shows what we have seen in the process of testing, what the numbers really represent and what the system is failing to meet when a production deployment is involved.

6.1 System Performance Evaluation

The initial thing we needed to check was a mere fact that the system was capable of sustaining a constant stream of data feed without lagging or collapsing. A monitoring system that comes with a huge amount of latency between a sensor reading and its display on the dashboard is not useful in practice - by the time an engineer notices the alert the situation might have changed.

During testing, the Flask backend accepted every incoming reading and responded within a consistently brief period of time regardless of us raising the frequency of data to levels greater than the standard two-second interval. The pipeline validation + normalization + ML prediction + threshold check + database write + alert dispatch have been run through all the way to the end without this observed lag under usual testing circumstances. The polling cycle on the dashboard showed the new values as soon as possible, that is, the live graphs were always kept up-to-date and did not show a delay with the real sensor values.

Scale Database performance was better than we thought it would be. Our test system used a number of thousands of records in the SensorReading table and query response times of the historical filtering and trend visualization features were fast enough to make them feel responsive in the dashboard. This was helped on by the three indexes on timestamp, alert_level and is an anomaly, without which, queries on thousand records or more would be slowed significantly.

The system has been found to maintain good stability during long testing periods without increasing or decreasing memory or slowing down response times and this is

the reason it is appropriate to the type of continuous long-running use that structural monitoring demands.

6.2 Anomaly Detection Results

To ensure that the logic of classification acted as expected throughout the range, the anomaly detection module was exercised in three conditions based on simulated conditions of normal operation, moderate stress, and extreme stress.

In normal conditions, all the sensor parameters remained in safe ranges, and the system always gave the status of Normal. The dashboard displayed green readings, no warning signals were produced, and the ML anomaly scores were nearer to zero or positive — which confirms that the Isolation Forest model identified these scores properly as they were in line with the normal behavior that it learned.

Parameter	Normal Range	Warning Threshold	Critical Threshold
Vibration	0.1 — 2.0 units	Above 2.0 units	Above 2.5 units
Strain	0.0 — 0.5 units	Above 0.5 units	Above 0.7 units
Temperature	20°C — 35°C	Above 35°C	Above 40°C

Table 6.1: Anomaly Detection Threshold Values and Classification

When moderate deviations were added in, strain values near to, but not on the 0.7 Critical threshold, or vibration near to, but not on the 2.02.3 range, the system made correct labels of Warning. The threshold layer signaled the boundary crossings and the ML layer generated that produced moderately negative scores of anomaly in respect to statistical deviation of normal cluster. This happens to be the range of most practical importance of detection in that early intervention can be made before things get risky.

In case of extreme values were simulated, i.e. vibration over 2.5, strain over 0.7, or temperature over 40C, the system automatically recognized them as Critical and turned on the dashboard red and directed the alert dispatch pipeline. The ML anomaly scores of such readings were very negative, usually between the range of -0.04 and -0.09, which proves that the model alone was in agreement with the threshold based classification.

One finding worth mentioning: the ML model and threshold system do not necessarily coincide and that is more or less on purpose. We noticed some readings of the Isolation Forest which were noted as statistically queer, i.e. negative scores, but which did not violate some threshold. These were instances where the trend in values of the three sensors combined was odd, although none of them went beyond its threshold. These are the cases which a system based on a threshold system would not detect at all and the dual-layer approach picks them up.

The ML models also retrained well on testing. Once enough data was collected, retrain if needed method automatically ran and trained a fresh Isolation Forest model using the latest readings and changed the model without any downtime in the system. The new model showed the same predictions as the old one with the normal readings and consequently sensitive enough in the case of abnormal readings.

6.3 Visualization and Monitoring Output

The dashboard has worked well since it is the main interface used to display the monitoring results. During testing, there were three observations that were made prominent:

And, to start with, the live graphs provided a truly practical view of structural behavior through time - not only momentary values. The 20-point window was rolling in nature so that you could determine if a spike was isolated or a continued trend and that is important in differentiating an actual structural issue versus a temporary sensor variation.

Fig 6.1: Real-Time Structural Monitoring Dashboard

Second, the color severity pointers of amber (warning), red (critical) were extended to the status cards and the outer borders of the graphs, which allowed the overall system state to be immediately glanced without going through the numbers. The alert state was always visual during the testing, several seconds after the anomaly had taken place.

Fig 6.2: Real-Time Sensor Trend Visualization

Historical Readings

ID	Timestamp	Vibration	Strain	Temperature	Anomaly	Alert Level	Score
1133	2/24/2026, 2:12:12 PM	2.2	0.58	24.4°C	Normal	warning	0.000
1132	2/24/2026, 2:11:34 PM	0.77	0.364	30.6°C	Normal	normal	0.000
1131	2/24/2026, 2:11:32 PM	0.76	0.156	22.2°C	Normal	normal	0.000
1130	2/24/2026, 2:11:31 PM	1.49	0.368	23.4°C	Normal	normal	0.000
1129	2/24/2026, 2:11:31 PM	0.66	0.114	22.3°C	Normal	normal	0.000
1128	2/24/2026, 2:11:31 PM	1.03	0.521	23.5°C	Normal	warning	0.000
1127	2/24/2026, 2:11:24 PM	1.72	0.596	22.9°C	Normal	warning	0.000
1126	2/24/2026, 2:11:24 PM	1.58	0.655	31°C	Normal	warning	0.000
1125	2/24/2026, 2:11:23 PM	1.37	0.234	28.8°C	Normal	normal	0.000
1124	2/24/2026, 2:11:23 PM	0.7	0.626	24.8°C	Normal	warning	0.000
1123	2/24/2026, 2:11:16 PM	2.16	0.239	25°C	Normal	warning	0.000
1122	8/5/2025, 1:26:49 PM	1.65	0.527	23.4°C	Normal	warning	0.099
1121	8/5/2025, 1:26:47 PM	1.83	0.763	24.8°C	Normal	critical	0.097
1120	8/5/2025, 1:26:45 PM	2.38	0.324	29.8°C	Normal	warning	0.073

1 2 3 Next

Fig 6.3: Historical Sensor Readings and Alert Classification

Third, the historical filtering option on the /history page was found to be a more practically useful aspect of the entire system. Having the ability to fetch all Critical readings within a given time period, export to CSV, and view the respective ML anomaly scores in Excel is precisely the type of workflow that a real maintenance engineer would employ in order to investigate an incident post-factum. The badges of anomaly and level of alerts in the table of history were coded in different colors which meant that scanning patterns was easy without having to read each and every value of the data.

Connection status indicator, which shows the last update time when connected and a reconnect message when the data feed fails was a minor feature, but it came in handy during testing when we intentionally cut the data feed. The dashboard rendered the disruption at once as opposed to displaying stale information in silence.

6.4 System Responsiveness and Reliability

In addition to the functional output we also considered the overall behavior of the system over a sustained operation - since a monitoring system that operates one hour and then fails over days is not really very useful.

The Flask backend could be run over long periods of time. The use of memory remained constant with time- there was no build up that indicated memory leakages in the processing pipeline. Despite the increasing number of records, the database remained fast enough to prove that the indexed SQLite configuration is suitable in a single-structure monitoring context that this system will be used in.

The modular service system, in which sensor creation, ML prediction, alert delivery, and statistics were represented as independent services, ensured that the other service was not impacted when we changed one in the course of testing. This was especially obvious during the testing of the ML retraining: changing the working model during the operation was not evidenced by any obvious changes on the dashboard or alert mechanism.

The alert fatigue protection has acted appropriately on all test conditions. The system would rightly target SMS and email transmissions to five per hour instead of drowning the recipient with hundreds of the same message during intervals of sustained anomalies in which Critical readings were being generated continuously.

6.5 Discussion

The findings affirm the software-first approach as viable and can deliver a system that is good in practice, and not merely in theory. There are some observations that should be discussed honestly.

The threshold rule combined with Isolation Forest ML that was called the dual-layer anomaly detector was as covering as either of the two would have been. Obvious anomalies were directly classified instantly and in a transparent manner by the threshold layer. The ML layer identified small multi-parameter outliers which would not have been detected individually by any threshold. With such a combination, in a real deployment, there is less missed anomaly without a significant increase in false alarm rates.

One of the features that we would mention as practically significant is the 24-hour automatic retraining model. The models based on the techniques of the static anomaly detection deteriorate over time as the pattern of the structural behavior change seasonal temperature variations, traffic loads changes, and gradual settlement of the material. A model that had been trained and it was left to run would ultimately begin giving false baseline estimates. This is dealt with through the retraining mechanism.

With that said, there are limitations which we would like to get straight. The system has not been tested using real hardware and thus, it is not known how it will integrate with physical sensors. It requires the ML models to have at least 50 readings before they can be trained, i.e. during initial deployment there exists a cold-start period, during which threshold detection is the only active one. The contamination parameter -1.00 - -1.0 An actual deployment of a production would be advantaged by having this parameter tuned, depending on the observed false positive and false negative rates in the particular structure under monitoring.

SQLite worked well at this scale in our testing environment, although a system where many structures are being tracked or storing years of consecutive measurements would have to be moved over to PostgreSQL or a cloud-native database. This migration is easy using the SQLAlchemy ORM, however, this is a scaling consideration since the current single-structure situation is being considered.

All in all, the findings indicate that the platform is doing what it promises to do - a realistic, deployable system of SHM that moves beyond simulation-based investigations and into the actual world of infrastructure measurement, and the software architecture is truly prepared to take the next step of moving to the next level of integrating IoT sensors effectively.

CHAPTER 7

TESTING AND VALIDATION

7.1 Introduction to Testing

Testing and validation constitute an important phase of the software development lifecycle, which makes sure that every element of the system is functioning as expected, both under normal and boundary conditions. In the current project, the testing was conducted in three levels i.e. unit testing, integration testing and system testing and then validation phase was done which was used to determine whether the system had achieved its stated objectives. As the platform is a constant monitoring system, a particular focus was made on ensuring that there would be no one component failure resulting in silent errors in the pipeline.All of the testing was done in a Windows 10 development environment on Python 3.10, flask, SQLite, scikit-learn. The test scenarios were to include the normal operating conditions, boundaries near the threshold limits, and extreme anomalous conditions.

7.2 Unit Testing

Unit testing was done on each unit separately to check that the data generation code had an advantage over real work condition and was able to create reasonable and properly structured outputs.

7.2.1 Module 1 — Sensor Simulation Unit Tests

The sensor_service.py module was tested independently to verify that the data generation logic produced realistic and correctly structured outputs.

Table 7.1: Unit Test Cases — Sensor Simulation Module

Test Case ID	Test Description	Input	Expected Output	Actual Output	Result
UT-SS-1	Temperature follows daily sinusoidal pattern	hour = 12 (noon)	Temperature near 33°C (peak)	32.7°C	Pass
UT-SS-2	Temperature at night is lower	hour = 0 (midnight)	Temperature near 17°C (trough)	17.4°C	Pass
UT-SS-3	Vibration derived from temperature	temperature = 30°C	Vibration near 1.8 units	1.76 units	Pass

UT-SS-4	Vibration never goes below minimum	Any input	Vibration >= 0.1	0.1 (clipped)	Pass
UT-SS-5	Strain generated independently	Any input	Strain between 0.0 and 0.8	0.43	Pass
UT-SS-6	Spike generation at 8% probability	1000 readings generated	~80 vibration spikes	83 spikes	Pass
UT-SS-7	JSON payload contains all required fields	Single reading generated	Keys: vibration, strain, temperature, timestamp	All keys present	Pass
UT-SS-8	Timestamp is UTC formatted	Single reading generated	ISO format UTC string	Correct format	Pass

Unit tests of sensor simulations were all successful. The values of the output were reported as per expectations and a JSON payload format was also ensured to be as per the format expected by the Flask ingestion endpoint.

7.2.2 Module 2 anomaly detection unit tests

The ml service py ml anomaly detection logic and Flask route threshold checking were tested separately.

Table 7.2: Unit Test Cases — Threshold Detection

Test Case ID	Test Description	Input Values	Expected Alert Level	Actual Alert Level	Result
UT-TD-1	All values within normal range	Vib=1.5, Strain=0.3, Temp=28°C	Normal	Normal	Pass
UT-TD-2	Vibration at Warning boundary	Vib=2.0, Strain=0.3, Temp=28°C	Warning	Warning	Pass
UT-TD-3	Vibration above Critical threshold	Vib=2.6, Strain=0.3, Temp=28°C	Critical	Critical	Pass

UT-TD-4	Strain at Warning boundary	Vib=1.5, Strain=0.5, Temp=28°C	Warning	Warning	Pass
UT-TD-5	Strain above Critical threshold	Vib=1.5, Strain=0.75, Temp=28°C	Critical	Critical	Pass
UT-TD-6	Temperature at Warning boundary	Vib=1.5, Strain=0.3, Temp=35°C	Warning	Warning	Pass
UT-TD-7	Temperature above Critical threshold	Vib=1.5, Strain=0.3, Temp=41°C	Critical	Critical	Pass
UT-TD-8	Multiple sensors in Warning range	Vib=2.1, Strain=0.55, Temp=36°C	Warning	Warning	Pass
UT-TD-9	Mixed Critical and Normal sensors	Vib=2.6, Strain=0.3, Temp=28°C	Critical	Critical	Pass
UT-TD-0	All values exactly at Normal upper limit	Vib=1.99, Strain=0.49, Temp=34.9°C	Normal	Normal	Pass

Table 7.3: Unit Test Cases — ML Anomaly Detection

Test Case ID	Test Description	Input	Expected Outpu	Actual Output	Result
UT-ML-01	Model training with sufficient data	100 normal readings	Model trained successfully, is_active = True	Model trained, active	Pass
UT-ML-02	Prediction on clearly normal reading	Vib=1.2, Strain=0.2, Temp=25°C	is_anomaly = False, score > 0	False, score = 0.041	Pass

UT-ML-03	Prediction on extrem anomalous reading	Vib=3.5, Strain=0.9, Temp=45°C	is_anomaly = True, score < 0	True, score = -0.072	Pass
UT-ML-04	Model falls back gracefully before training	No trained model available	Returns False, 0.0 — no crash	False, 0.0	Pass
UT-ML-05	Retraining triggers after 24 hours	Model age > 86400 seconds	New model created, old deactivated	New model active	Pass
UT-ML-06	Retraining skipped if model is recent	Model age < 86400 seconds	No retraining triggered	No retraining	Pass
UT-ML-07	Retraining skipped if insufficient data	Less than 50 readings	No retraining triggered	No retraining	Pass
UT-ML-08	StandardScaler applied before prediction	Raw multi-scale inputs	Normalized features used	Normalization confirmed	Pass

7.2.3 Module 3- Alert System Module Tests.

The alert service was tested alone with fake sensor readings to make sure dispatch logic and alert fatigue protection.

Table 7.4: Unit Test Cases — Alert System

Test Case ID	Test Description	Input Condition	Expected Behaviour	Actual Behaviour	Result
UT-AL-1	Email triggered on Warning reading	Alert level = Warning	Email dispatch attempted	Email dispatched	Pass
UT-AL-2	Email and SMS triggered on Critical reading	Alert level = Critical	Both email and SMS dispatched	Both dispatched	Pass
UT-AL-3	No alert on Normal reading	Alert level = Normal	No dispatch	No dispatch	Pass

UT-AL-4	Alert fatigue — 5 alerts within 1 hour	6th alert of same severity within 60 min	6th alert suppressed	Suppressed	Pass
UT-AL-5	Alert fatigue resets after 1 hour	6th alert after 60 min window	Alert dispatched	Dispatched	Pass
UT-AL-6	All alert attempts logged to AlertLog	Any dispatch attempt	Record written to AlertLog table	Record written	Pass
UT-AL-7	Failed dispatch still logged	SMTP credentials invalid	Failure logged with error	Logged	Pass

7.3 Integration Testing

Integration testing ensured that the individual components were integrated properly into a linked pipeline. All the test scenarios were executed on the full system and had all its components running concurrently.

7.3.1 Ingestion to Detection Integration

This test confirmed that sensor data that emanates through the simulation layer into the anomaly detection layer is directed where it belongs and preprocessing is done followed by prediction.

Table 7.5: Integration Test Cases — Ingestion to Detection Pipeline

Test Case ID	Test Description	Expected Behaviour	Result
IT-01	Sensor reading sent via HTTP POST to /sensor-data endpoint	Flask receives payload, validates, passes to preprocessing	Pass
IT-02	Missing field in JSON payload	Request rejected with 400 error, not passed to ML	Pass
IT-03	Valid reading processed through full pipeline	Validation → Normalization → ML prediction → Threshold check → DB write	Pass
IT-04	ML anomaly flag and threshold alert level both stored in same database record	Single SensorReading record contains both results	Pass

IT-05	Normal reading does not trigger alert dispatch	AlertLog remains unchanged for Normal readings	Pass
IT-06	Critical reading triggers both DB write and alert dispatch in same cycle	Record written and alert sent within same request cycle	Pass

7.3.2 Database Integration Testing

These tests ensured that the data that the processing layer had written was able to be read by the dashboard layer accurately.

Table 7.6: Integration Test Cases — Database Operations

Test Case ID	Test Description	Expected Behaviour	Result
IT-DB-01	Sensor reading written to SensorReading table	All fields persisted correctly including ML score	Pass
IT-DB-02	AlertLog record references correct SensorReading ID	Foreign key relationship maintained correctly	Pass
IT-DB-03	Historical query filtered by timestamp range	Returns only records within specified date range	Pass
IT-DB-04	Historical query filtered by alert_level = Critical	Returns only Critical records	Pass
IT-DB-05	Historical query filtered by is_anomaly = True	Returns only ML-flagged anomaly records	Pass
IT-DB-06	CSV export contains correct headers and all filtered records	Downloaded CSV matches database query results	Pass
IT-DB-07	Indexes on timestamp, alert_level, is_anomaly speed up filtering queries	Queries on 5000+ records complete in under 1 second	Pass

7.3.3 Dashboard Backend Integration Testing

This was a test of the live polling between the browser dashboard and the Flask backend.

Table 7.7: Integration Test Cases — Dashboard to Backend

Test Case ID	Test Description	Expected Behaviour	Result
IT-DB-01	Dashboard polls /sensor-data every 2 seconds	New reading generated and returned on each poll	Pass
IT-DB-02	Live graphs update without page reload	Chart.js receives new data point and updates chart	Pass
IT-DB-03	Warning reading turns status cards amber	Card border and value text change to amber color	Pass
IT-DB-04	Critical reading turns status cards red with pulse	Card border animates red pulse	Pass
IT-DB-05	Alert banner appears on Warning/Critical	Banner displayed with correct severity message	Pass
IT-DB-06	Alert banner auto-dismisses after 10 seconds	Banner disappears automatically	Pass
IT-DB-07	Connection lost message shown on polling failure	Status bar shows reconnection message	Pass
IT-DB-08	Connection restored message shown on recovery	Status bar reverts to Connected with timestamp	Pass

7.4 System Testing

System testing was done to test the entire platform as a unified entity in its sustained conditions in a simulated real-world monitoring situation.

7.4.1 Continuous Operation Test

An extensive time was taken to run the system continuously with sensor simulation providing readings of 2 seconds. This was to ensure that stability was checked, memory leaks were also checked and also to ensure that performance did not deteriorate as time went by.

Table 7.8: System Test — Continuous Operation

Metric	Observation	Outcome
Duration of continuous run	4 hours	Stable throughout
Total readings generated	Approximately 7,200 readings	All processed successfully
Memory usage over time	Flat — no growth detected	No memory leak
Average response time per reading	Under 200 milliseconds	Within acceptable limits
Dashboard polling failures	0	Reliable data feed
Database query response time at 7,200 records	Under 1 second	Acceptable performance

7.4.2 Testing of Anomaly Detection System

This test confirmed the end to end anomaly detection and alerting behaviour in a variety of simulated conditions.

Table 7.9: System Test — Anomaly Detection Scenarios

Scenario	Simulated Condition	Expected System Response	Actual System Response	Result
ST-AD-01	All sensors within normal range for 30 minutes	Normal status, no alerts, green dashboard	Normal status maintained, no alerts	Pass
ST-AD-02	Vibration spike to 2.8 (Critical)	Immediate Critical alert, red dashboard, email + SMS	Critical detected within 2 seconds, alerts sent	Pass
ST-AD-03	Strain gradually rising from 0.3 to 0.6 over 10 minutes	Warning alert when crossing 0.5	Warning triggered at crossing point	Pass

ST-AD-04	Temperature rising above 35°C due to simulated midday	Warning alert triggered	Warning triggered correctly	Pass
ST-AD-05	Sustained Critical condition for 15 minutes	Maximum 5 alerts per hour — fatigue protection active	Only 5 alerts sent i first hour	Pass
ST-AD-06	ML model detects multi-sensor anomaly below individual thresholds	is_anomaly = True with negative anomaly score	ML flagged correctly while threshold showed Normal	Pass
ST-AD-07	System recovers to Normal after anomaly clears	Dashboard returns to green, no further alerts	Normal status restored	Pass

7.4.3 Retraining System Test ML Model

This test confirmed the automated retraining mechanism in the actual working conditions.

Table 7.10: System Test — ML Model Retraining

Test Step	Expected Behaviour	Actual Behaviour	Result
System starts with no trained model	Falls back to threshold-only detection	Threshold detection active, no ML crash	Pass
50 readings accumulated	Retraining check passes minimum sample condition	Model training triggered	Pass
New model trained	MLModel record created with is_active = True	Record created correctly	Pass
Old model after 24 hours	retrain_if_needed deactivates old model	Old model marked is_active = False	Pass
New model trained on last 7 days of data	New MLModel record created with updated trained_at timestamp	New active model confirmed	Pass

Predictions after retraining	Consistent anomaly detectio with new model	Predictions aligned with expected behavior	Pass

7.5 Performance Testing

Performance testing was done to test the way the system could withstand higher volumes of data and simultaneous conditions.

Validity against Objectives The test's validity can be assessed by comparing it with external criteria like item scores, evaluations, and assessments.

Table 7.11: Performance Test Results

Test Scenario	Condition	Measured Result	Acceptable Threshold	Status
Single reading processing time	Standard 2-second interval	< 200 ms	< 500 ms	Pass
Dashboard update latency	New reading to graph update	< 2.5 seconds	< 5 seconds	Pass
Database write time	Per reading insert with ORM	< 50 ms	< 200 ms	Pass
Historical query — 1,000 records	Unfiltered fetch	< 300 ms	< 1 second	Pass
Historical query — 5,000 records	Filtered by alert_level	< 800 ms	< 2 seconds	Pass
CSV export — 1,000 records	Full dataset export	< 1 second	< 3 seconds	Pass
Memory usage after 4 hours	Continuous operation	Stable, no growth	No memory leak	Pass

7.6 Cross-Checking With Objectives

The validity of the test is possible when the test is compared with other external measures such as item scores, evaluation and assessment.

This section justifies the success of the system on meeting each of the objectives listed in Chapter 1.

Table 7.12: Validation Against Project Objectives

Objective	**Validation Method**	**Outcome**
Develop a software-first SHM platform without requiring hardware	System ran entirely on simulated data with full pipeline operational	Achieved
Simulate realistic vibration, strain, and temperature sensor behavior	Sensor outputs validated against real-world SHM parameter ranges	Achieved
Implement dual-layer anomaly detection combining threshold rules and ML	Both detection layers operational; ML caught multi-parameter anomalies missed by thresholds	Achieved
Build a Flask REST API backend with modular service architecture	Blueprint-based architecture with separate services for ML, alerts, and sensors	Achieved
Design a usable web dashboard with live graphs and historical filtering	Dashboard tested with live data — real-tim graphs, color-coded alerts, CSV export all functional	Achieved
Implement automated alerts wi fatigue protection	Email and SMS alerts working; fatigue lim of 5 per hour verified in system testing	Achieved
Store historical data with trend analysis capability	SQLite with SQLAlchemy ORM, indexed queries, historical filter page operational	Achieved
Ensure IoT-ready architecture with hardware-agnostic ingestion	REST API accepts same JSON format as ESP32/Arduino; no backend changes neede for hardware swap	Achieved

Eight project objectives were also verified and authenticated as being met under the context of simulated data testing.

7.7 Limitations that were known in the testing process

The limitations that had been identified during the testing phase include the following and are admitted as the areas of improvement in the future.

Before 50 readings have been gathered, the ML detection layer is off and the detection is limited to threshold-based detection. This implies that anomalies that occur early below the threshold limits although statistically suspicious would be missed during early minutes of deployment.

The parameter of the Isolation Forest model that determines the contamination was taken as 0.1 according to the general best practice. This value had not been empirically optimized with against real structural data. Calibration of this parameter would be needed in a deployment of production deployment, depending on the rates of false positives and false negatives of the particular structure under monitoring.

Simulated data were used to provide all testing. Although the simulation was supposed to be based on realistic structural sensor patterns, the real IoT hardware added new variables such as sensor drift, packet loss, timing irregularities and calibration offsets which were not possible to simulate in the current test environment.

The single-structure single-user testing of this project was conducted and SQLite worked well. Nonetheless, it has had its limitations on large data volume and access by large number of concurrent users. An environment that tracks multiple structures at the same time would need to be migrated to PostgreSQL or a cloud-native database.

7.8 Summary

Unit, integration, system, and performance level testing in all three modules and full-end-to-end pipeline was tested. The formal test cases that were run throughout the testing phases were 47 in number with the final test cases giving a Pass result. The system exhibited reliable and consistent behaviour with continuous work, properly identified anomalies of all levels, preserved alert fatigue and conducted automated retraining of the ML models, and displayed the results in a transparent way using the live dashboard. The validation exercise ensured that the eight project objectives mentioned in Chapter 1 were met within the frame of the present implementation.

CHAPTER 8

CONCLUSION

In hindsight , the project fulfilled the intended objective: it provided a working, software first structural health monitoring system that processes sensor data in real-time, detects anomalies, and displays all information in an easily readable form in a live dashboard. That may seem simple, though, it requires more head than the completed mechanism makes it seem to get every component to work together in a clean manner.

The Flask backend was well-performing. Ingesting data, validating it, preprocessing, and detecting anomalies all consume time without the sort of latency that will make a real time monitoring system irritating to use. The threshold-based classification, which was Normal, Warning, Critical, provided the system with a very transparent logic, which anyone can easily comprehend without having to be informed about what is going on in the background. That matters in practice. A monitoring system that generates results that can not be understood by anyone is not really helpful.

The item we are most likely to be content with was the dashboard. Taking sensor readings and storing them in a database is one thing, and displaying them in a manner that is useful in actually aiding an engineer in making a decision is another. The live graphs, color-coded alert indicators, historical filtering and CSV export all combined into something that seems usable instead of technically functional.

The focus on the software aspect makes this project stand out of many studies of SHM. The majority of literature in this area will go directly to sensors or algorithms. We did it right initially on the architecture with modular layers, clean APIs, and good data-handling and it can be seen in the behavior of the system. It is also stable, very easy to follow, and it is actually prepared to be further developed with the next step which is to join real hardware, without having to redesign it.

We have limitations that we have been fair all through this report. The system has not been experimented with physical sensors. The ML models require warming up before

they can make predictions. The contamination parameter was not optimized on actual structural data. The system will require a replacement of SQLite before it can scale to the monitoring of multiple structures at the same time. All these are no longer surprises, they are the logical successive problems to address, rather than a weakness in the existing work.

What is more important here is the fact that the software-first approach can work in SHM. The issue with infrastructure safety is a fact and it is severe. The means to construct smart surveillance is available. The only thing that is often missing is a clean and deployable software layer that glues it all together and which is practically working. This is what this project was attempting to offer, and according to the outcomes, we believe that it is accomplishing it.

CHAPTER 9

FUTURE WORK

The existing system is not a finished product, it is a proven starting point - and it is on purpose. The architecture has been designed to expand, and there are a few evident directions of where the architecture should expand.

Real IoT sensor integration is the closest next step. It is already waiting to be ingested on the REST API ingestion layer. A simulation on Wi-Fi connected ESP32 with an accelerator and strain gauge would send the identical JSON format being sent by the simulation to the identical endpoints. The server would be unaware of the difference. The real work is the wiring of hardware and sensor calibration - the software is prepared. Actual structural data flowing through the system would also enable us to optimize the Isolation Forest contamination parameter according to observed behaviour instead of using defaults.

Upgrading the ML layer is the second priority. Isolation Forest and One-Class SVM are effective at finding statistical abnormalities, but they lack time-series behavior logic - i.e. the way the vibration gradually increases over the course of weeks before reaching a breakpoint, etc. This temporal reasoning is the reason why LSTM networks have been created. By incorporating an LSTM-based prediction layer with the already existing anomaly detection, the system would have prediction ability rather than a reactory detection ability.

Cloud deployment would make multi-structure monitoring available. At this point the system is localized and is capable of monitoring a single structure at a time. Migrating the backend to AWS, GCP, or Azure and using PostgreSQL instead of SQLite would enable one platform to handle dozens of structures at once, and all historical information can be accessed remotely. Such a migration can be easily done with the SQLAlchemy ORM it's a change in configuration rather than a code rewrite.

Expanded alerting is a rather minor addition that has immense practical effects. Twilio and Flask-Mail email and SMS are already partially applied to the codebase.

To be fully activated, credentials have to be configured, and delivery reliability must be tested. This can be further enhanced by adding mobile push notifications using Firebase Cloud Messaging, whereby field engineers would receive alerts in their phones without having to look at a dashboard.

A mobile interface is a system which would become truly accessible to individuals who are not seated in a desktop. Most of what a field engineer would require when off the dashboard would be in the form of a simple React Native or Flutter application that displays live sensor graphs, the current alert status, and a history of recent anomalies.

GIS integration is a more long-term direction, though an attractive one. In the case of smart city applications in which dozens of bridges, buildings, and other public structures are under observation, a map-based interface displaying the current health status of each structure at a glance would be much more viable than a list of dashboards.

Security hardening is of concern on the moment when the system manages actual infrastructure data. The existing implementation lacks authentication of API endpoints as well as encryption of data in transit. They both should be done prior to any actual implementation, including HTTPS to perform communication with the API, JWT-based authentication to access the dashboard and encrypted database storage of sensitive structural data.

All these improvements are feasible on the existing architecture as the base. The modular structure enables them to be added during the process one at a time without destroying what is already working that is the whole concept of developing the software layer with lots of care before adding everything else above it.

REFERENCES

[1] C. R. Farrar and K. Worden, “An introduction to structural health monitoring,” *Philosophical Transactions of the Royal Society A*, vol. 365, no. 1851, pp. 303–315, 2007.

[2] J. P. Lynch and K. J. Loh, “A summary review of wireless sensors and sensor networks for structural health monitoring,” *Shock and Vibration Digest*, vol. 38, no. 2, pp. 91–128, 2006.

[3] H. Li, J. Ou, and Z. Zhou, “Applications of Internet of Things in structural health monitoring,” *Smart Structures and Systems*, vol. 15, no. 2, pp. 381–401, 2015.

[4] X. W. Ye, Y. H. Su, and J. P. Han, “Structural health monitoring based on machine learning techniques: A survey,” *Structural Health Monitoring*, vol. 18, no. 1, pp. 1–23, 2019.

[5] E. P. Carden and P. Fanning, “Vibration based condition monitoring: A review,” *Structural Health Monitoring*, vol. 3, no. 4, pp. 355–377, 2004.

[6] K. Worden and J. M. Dulieu-Barton, “An overview of intelligent fault detection in systems and structures,” *Structural Health Monitoring*, vol. 3, no. 1, pp. 85–98, 2004.

[7] S. W. Doebling, C. R. Farrar, and M. B. Prime, “A summary review of vibration-based damage identification methods,” *Shock and Vibration Digest*, vol. 30, no. 2, pp. 91–105, 1998.

[8] H. Sohn, C. R. Farrar, F. M. Hemez, and D. D. Shunk, “A review of structural health monitoring literature: 1996–2001,” Los Alamos National Laboratory Report, 2004.

[9] H. Sun, H. V. Burton, and H. Huang, “Smart infrastructure monitoring using cloud-based structural health monitoring systems,” *Journal of Infrastructure Systems*, vol. 26, no. 2, 2020.

[10] Y. Bao and H. Li, “Machine learning paradigms for structural health monitoring,” *Structural Health Monitoring*, vol. 18, no. 5–6, pp. 1369–1389, 2019.

PLAGARISM REPORT

turnitin Page 1 of 132 - Cover Page Submission ID trn:oid:::1:3523120665

2 2

SOFTWARE-FIRST STRUCTURAL HEALTH MONITORING (SHM) PLATFORM FOR REAL TIME ANOMALY DETECTION

conference paper

AI&DS DEPARTMENT

Panimalar Engineering College

Document Details

Submission ID

trn:oid:::1:3523120665

Submission Date

Mar 30, 2026, 4:06 PM GMT+5:30

Download Date

Mar 30, 2026, 4:34 PM GMT+5:30

File Name

Report_AI.pdf

File Size

2.8 MB

120 Pages

29,100 Words

156,845 Characters

turnitin Page 1 of 132 - Cover Page Submission ID trn:oid:::1:3523120665

turnitin Page 2 of 132 - Integrity Overview Submission ID trn:oid:::1:3122120665

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Quoted Text

Match Groups

- **186** Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- **1** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
6% Publications
6% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

turnitin Page 2 of 132 - Integrity Overview Submission ID trn:oid:::1:3122120665

AI SIMILARITY REPORT

2 2

SOFTWARE-FIRST STRUCTURAL HEALTH MONITORING (SHM) PLATFORM FOR REAL TIME ANOMALY DETECTION

conference paper

AI&DS DEPARTMENT

Panimalar Engineering College

Document Details

Submission ID

trn:oid:::1:3522120665

Submission Date

Mar 30, 2026, 4:06 PM GMT+5:30

Download Date

Mar 30, 2026, 4:34 PM GMT+5:30

File Name

Report_A1.pdf

File Size

2.8 MB

120 Pages

29,100 Words

158,845 Characters

turnitin Page 1 of 122 - Cover Page Submission ID trn:oid:::1:3522120665

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

PUBLICATION DESCRIPTION

Publisher: ASEAN Journal of Scientific and Technological Reports (AJSTR)

Paper Title: Software-First Structural Health Monitoring (SHM) Platform for Real-Time Anomaly Detection

Authors: Dr. S. Chakravarathi, P. R. Gopikashree, M. Devadarshini, S. Dhivyadharshini

Status: Paper Submitted

PAPER SUBMISSION MAIL

Gmail

Gopikashree PR <gopikassakipog@gmail.com>

[AJSTR] Submission Acknowledgement

1 message

Sompong O-Thong via Thai Journals Online (ThaiJO) <admin@tci-thaijo.org> Sat, Mar 28, 2026 at 11:18 AM
Reply-To: Sompong O-Thong <sompong@tsu.ac.th>
To: Gopikashree P R <gopikassakipog@gmail.com>

Gopikashree P R:

Thank you for submitting the manuscript, "Software-First Structural Health Monitoring (SHM) platform for Real Time Anomaly Detection" to ASEAN Journal of Scientific and Technological Reports. With the online journal management system that we are using, you will be able to track its progress through the editorial process by logging in to the journal web site:

Submission URL: https://ph02.tci-thaijo.org/index.php/tsujournal/authorDashboard/submission/264610
Username: gopikashreepr

If you have any questions, please contact me. Thank you for considering this journal as a venue for your work.

Sompong O-Thong

APPENDIX

app.py — Main Application Entry Point

```
from flask import Flask
from models import db
from routes.main import main_bp
from routes.api import api_bp
from routes.history import history_bp
from config import Config
app = Flask(__name__)
app.config.from_object(Config)
db.init_app(app)
app.register_blueprint(main_bp)
app.register_blueprint(api_bp)
app.register_blueprint(history_bp)
with app.app_context():
    db.create_all()
if __name__ == '__main__':
    app.run(debug=True)
```

config.py — Configuration Settings

```
class Config:
    SQLALCHEMY_DATABASE_URI = 'sqlite:///shm.db'
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    SECRET_KEY = 'shm-secret-key'
    VIBRATION_WARNING = 2.0
    VIBRATION_CRITICAL = 2.5
    STRAIN_WARNING = 0.5
    STRAIN_CRITICAL = 0.7
    TEMP_WARNING = 35.0
    TEMP_CRITICAL = 40.0
    ALERT_FATIGUE_LIMIT = 5
```

```
    ALERT_FATIGUE_WINDOW = 3600
    ML_RETRAIN_INTERVAL = 86400
    ML_MIN_SAMPLES = 50
    ML_LOOKBACK_DAYS = 7
    ML_CONTAMINATION = 0.1
```

models.py — Database Models

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime
db = SQLAlchemy()
class SensorReading(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    timestamp = db.Column(db.DateTime, default=datetime.utcnow)
    vibration = db.Column(db.Float)
    strain = db.Column(db.Float)
    temperature = db.Column(db.Float)
    is_anomaly = db.Column(db.Boolean, default=False)
    anomaly_score = db.Column(db.Float, default=0.0)
    alert_level = db.Column(db.String(20), default='normal')
    alert_sent = db.Column(db.Boolean, default=False)
    alerts = db.relationship('AlertLog', backref='reading')
class MLModel(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    algorithm = db.Column(db.String(50))
    trained_at = db.Column(db.DateTime, default=datetime.utcnow)
    sample_count = db.Column(db.Integer)
    contamination = db.Column(db.Float)
    is_active = db.Column(db.Boolean, default=False)
class AlertLog(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    sensor_reading_id = db.Column(db.Integer, db.ForeignKey('sensor_reading.id'))
    alert_type = db.Column(db.String(20))
    recipient = db.Column(db.String(100))
```

```
    sent_at = db.Column(db.DateTime, default=datetime.utcnow)
    success = db.Column(db.Boolean, default=False)
```

services/sensor_service.py — Sensor Simulation

```
import math, random
from datetime import datetime, timezone
def generate_sensor_reading():
    hour = datetime.now().hour
    base_temp = 25 + 8 * math.sin(math.pi * hour / 12)
    temperature = round(base_temp + random.uniform(-5, 5), 2)
    vibration = round(max(0.1, temperature * 0.06 +
                    random.uniform(-1.5, 1.5)), 2)
    strain = round(random.uniform(0.0, 0.8), 3)
    if random.random() < 0.08:
        vibration = round(random.uniform(2.3, 3.5), 2)
    if random.random() < 0.08:
        strain = round(random.uniform(0.65, 0.95), 3)
    return {
        'vibration': vibration,
        'strain': strain,
        'temperature': temperature,
        'timestamp': datetime.now(timezone.utc).isoformat()
    }
```

services/ml_service.py — ML Anomaly Detection

```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from datetime import datetime, timedelta
from models import db, SensorReading, MLModel
import numpy as np
def get_features(reading):
    hour = reading.timestamp.hour
```

```
    dow = reading.timestamp.weekday()
    return [reading.vibration, reading.strain,
            reading.temperature, hour, dow]
def predict_anomaly(reading):
    active = MLModel.query.filter_by(is_active=True).first()
    if not active:
        return False, 0.0
    features = np.array([get_features(reading)])
    scaler = StandardScaler()
    features = scaler.fit_transform(features)
    model = IsolationForest(n_estimators=100,
                            contamination=active.contamination)
    score = model.decision_function(features)[0]
    prediction = model.predict(features)[0]
    return prediction == -1, round(float(score), 4)
def retrain_if_needed():
    active = MLModel.query.filter_by(is_active=True).first()
    if active:
        age = (datetime.utcnow() - active.trained_at).total_seconds()
        if age < 86400:
            return
    cutoff = datetime.utcnow() - timedelta(days=7)
    readings = SensorReading.query.filter(
        SensorReading.timestamp >= cutoff).all()
    if len(readings) < 50:
        return
    X = np.array([get_features(r) for r in readings])
    scaler = StandardScaler()
    X = scaler.fit_transform(X)
    model = IsolationForest(n_estimators=100, contamination=0.1)
    model.fit(X)
    if active:
        active.is_active = False
```

```
    new_model = MLModel(algorithm='IsolationForest',
                        sample_count=len(readings),
                        contamination=0.1, is_active=True)
    db.session.add(new_model)
    db.session.commit()
```

templates/dashboard.html — Main Dashboard Template

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Structural Health Monitoring Dashboard</title>
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
>
    <link rel="stylesheet" href="{{ url_for('static',
filename='css/style.css') }}">
</head>
<body>
    <nav class="navbar navbar-dark bg-dark px-4">
        <span class="navbar-brand fw-bold">Structural Health Monitoring
Dashboard</span>
        <div>
            <a href="/" class="btn btn-sm btn-outline-light me-2">Live
Dashboard</a>
            <a href="/history" class="btn btn-sm
btn-outline-light">History</a>
        </div>
    </nav>
    <div class="container-fluid mt-4 px-4">
        <div id="alert-banner" class="alert alert-danger d-none
alert-dismissible" role="alert">
            <span id="alert-message"></span>
            <button type="button" class="btn-close"
data-bs-dismiss="alert"></button>
```

```
</div>
<h4 class="text-center mb-1">Sensor Data Dashboard</h4>
<p class="text-center text-muted small mb-4">
    Real-time monitoring of vibration, strain, and temperature
</p>
<div class="row g-3 mb-4">
    <div class="col-md-4">
        <div class="card status-card" id="card-vibration">
            <div class="card-body text-center">
                <div class="text-muted small">Vibration</div>
                <div class="display-value" id="val-vibration">--</div>
                <div class="text-muted small">Units</div>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card status-card" id="card-strain">
            <div class="card-body text-center">
                <div class="text-muted small">Strain</div>
                <div class="display-value" id="val-strain">--</div>
                <div class="text-muted small">Units</div>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card status-card" id="card-temp">
            <div class="card-body text-center">
                <div class="text-muted small">Temperature</div>
                <div class="display-value" id="val-temp">--</div>
                <div class="text-muted small">°C</div>
            </div>
        </div>
    </div>
```

```
        </div>
        <div class="row g-3 mb-4">
            <div class="col-md-4">
                <div class="card p-3">
                    <div class="text-muted small mb-2">Vibration Data</div>
                    <canvas id="chart-vibration" height="120"></canvas>
                </div>
            </div>
            <div class="col-md-4">
                <div class="card p-3">
                    <div class="text-muted small mb-2">Strain Data</div>
                    <canvas id="chart-strain" height="120"></canvas>
                </div>
            </div>
            <div class="col-md-4">
                <div class="card p-3">
                    <div class="text-muted small mb-2">Temperature Data</div>
                    <canvas id="chart-temp" height="120"></canvas>
                </div>
            </div>
        </div>
        <div class="text-center text-muted small mt-2" id="connection-status">
            Connecting...
        </div>
    </div>
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min
.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <script src="{{ url_for('static', filename='js/chart.js') }}"></script>
</body>
</html>
```

templates/history.html — Historical Readings Template

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Historical Readings - SHM</title>

    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
>

    <link rel="stylesheet" href="{{ url_for('static',
filename='css/style.css') }}">

</head>

<body>

    <nav class="navbar navbar-dark bg-dark px-4">

        <span class="navbar-brand fw-bold">Structural Health Monitoring</span>

        <div>

            <a href="/" class="btn btn-sm btn-outline-light me-2">Live
Dashboard</a>

            <a href="/history" class="btn btn-sm
btn-outline-light">History</a>

        </div>

    </nav>

    <div class="container-fluid mt-4 px-4">

        <h5 class="mb-3">Historical Readings</h5>

        <div class="row g-2 mb-3">

            <div class="col-md-3">

                <input type="date" class="form-control form-control-sm"

                       id="filter-date-from" placeholder="From date">

            </div>

            <div class="col-md-3">

                <input type="date" class="form-control form-control-sm"

                       id="filter-date-to" placeholder="To date">

            </div>

            <div class="col-md-2">

                <select class="form-select form-select-sm"
id="filter-anomaly">
```

```
            <option value="">All readings</option>
            <option value="true">Anomalies only</option>
            <option value="false">Normal only</option>
        </select>
    </div>
    <div class="col-md-2">
        <select class="form-select form-select-sm" id="filter-alert">
            <option value="">All levels</option>
            <option value="normal">Normal</option>
            <option value="warning">Warning</option>
            <option value="critical">Critical</option>
        </select>
    </div>
    <div class="col-md-1">
        <button class="btn btn-sm btn-primary w-100"
                onclick="applyFilters()">Filter</button>
    </div>
    <div class="col-md-1">
        <button class="btn btn-sm btn-success w-100"
                onclick="exportCSV()">Export</button>
    </div>
</div>
<div class="table-responsive">
    <table class="table table-sm table-hover" id="readings-table">
        <thead class="table-dark">
            <tr>
                <th>ID</th>
                <th>Timestamp</th>
                <th>Vibration</th>
                <th>Strain</th>
                <th>Temperature</th>
                <th>Anomaly</th>
                <th>Alert Level</th>
```

```
                        <th>Score</th>
                    </tr>
                </thead>
                <tbody id="readings-body">
                </tbody>
            </table>
        </div>
        <div id="pagination" class="d-flex gap-2 mt-2"></div>
    </div>
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min
.js"></script>
    <script src="{{ url_for('static', filename='js/chart.js') }}"></script>
</body>
</html>
```

static/css/style.css — Dashboard Styles

```
body {
    background-color: #1a1a2e;
    color: #e0e0e0;
    font-family: 'Segoe UI', sans-serif;
}
.navbar {
    border-bottom: 1px solid #333;
}
.card {
    background-color: #16213e;
    border: 1px solid #0f3460;
    border-radius: 8px;
    color: #e0e0e0;
}
.status-card {
    transition: border-color 0.3s ease;
}
```

```
.status-card.warning {
    border-color: #ffc107;
}
.status-card.critical {
    border-color: #dc3545;
    animation: pulse-border 1s infinite;
}
@keyframes pulse-border {
    0%, 100% { border-color: #dc3545; }
    50%       { border-color: #ff6b6b; }
}
.display-value {
    font-size: 2rem;
    font-weight: 600;
    color: #a78bfa;
    margin: 4px 0;
}
.display-value.warning { color: #ffc107; }
.display-value.critical { color: #dc3545; }
.table {
    color: #e0e0e0;
    font-size: 0.85rem;
}
.table-dark th {
    background-color: #0f3460;
}
.badge-normal   { background-color: #198754; color: #fff; }
.badge-warning  { background-color: #ffc107; color: #000; }
.badge-critical { background-color: #dc3545; color: #fff; }
.badge-anomaly  { background-color: #7c3aed; color: #fff; }
#connection-status {
    font-size: 0.75rem;
    padding: 4px 0 12px;
```

```
}
.alert-banner-warning { background-color: #856404; border-color: #ffc107; }
.alert-banner-critical { background-color: #842029; border-color: #dc3545; }
```

static/js/chart.js — Dashboard JavaScript

```
const MAX_POINTS = 20;
const vibData  = { labels: [], datasets: [{ label: 'Vibration',
    data: [], borderColor: '#60a5fa', backgroundColor: 'rgba(96,165,250,0.1)',
    tension: 0.4, fill: true, pointRadius: 2 }] };
const strData  = { labels: [], datasets: [{ label: 'Strain',
    data: [], borderColor: '#fbbf24', backgroundColor: 'rgba(251,191,36,0.1)',
    tension: 0.4, fill: true, pointRadius: 2 }] };
const tempData = { labels: [], datasets: [{ label: 'Temperature',
    data: [], borderColor: '#34d399', backgroundColor: 'rgba(52,211,153,0.1)',
    tension: 0.4, fill: true, pointRadius: 2 }] };
const chartOpts = {
    responsive: true,
    animation: false,
    plugins: { legend: { display: false } },
    scales: {
        x: { ticks: { color: '#888', maxTicksLimit: 5, font: { size: 10 } },
             grid: { color: '#2a2a4a' } },
        y: { ticks: { color: '#888', font: { size: 10 } },
             grid: { color: '#2a2a4a' } }
    }
};
const vibChart  = new Chart(
    document.getElementById('chart-vibration'), {
        type: 'line', data: vibData, options: chartOpts });
const strChart  = new Chart(
    document.getElementById('chart-strain'), {
        type: 'line', data: strData, options: chartOpts });
const tempChart = new Chart(
```

```
    document.getElementById('chart-temp'), {
        type: 'line', data: tempData, options: chartOpts });
function addPoint(chartData, chart, label, value) {
    chartData.labels.push(label);
    chartData.datasets[0].data.push(value);
    if (chartData.labels.length > MAX_POINTS) {
        chartData.labels.shift();
        chartData.datasets[0].data.shift();
    }
    chart.update('none');
}
function setCardLevel(cardId, valId, value, level) {
    const card = document.getElementById(cardId);
    const val  = document.getElementById(valId);
    card.className = 'card status-card ' + (level !== 'normal' ? level : '');
    val.className  = 'display-value '    + (level !== 'normal' ? level : '');
    val.textContent = value;
}
function showAlertBanner(level, message) {
    const banner = document.getElementById('alert-banner');
    const msg    = document.getElementById('alert-message');
    if (level === 'normal') { banner.classList.add('d-none'); return; }
    banner.classList.remove('d-none', 'alert-danger', 'alert-warning');
    banner.classList.add(level === 'critical' ? 'alert-danger' :
'alert-warning');
    msg.textContent = message;
    setTimeout(() => banner.classList.add('d-none'), 10000);
}
async function fetchData() {
    try {
        const res  = await fetch('/sensor-data');
        const data = await res.json();
        const time = new Date(data.timestamp).toLocaleTimeString();
        addPoint(vibData,  vibChart,  time, data.vibration);
```

```
        addPoint(strData,  strChart,  time, data.strain);
        addPoint(tempData, tempChart, time, data.temperature);
        setCardLevel('card-vibration', 'val-vibration',
                     data.vibration,  data.alert_level);
        setCardLevel('card-strain',    'val-strain',
                     data.strain,     data.alert_level);
        setCardLevel('card-temp',      'val-temp',
                     data.temperature + '°C', data.alert_level);
        if (data.alert_level !== 'normal') {
            showAlertBanner(data.alert_level,
                data.alert_level.toUpperCase() +
                ' — Anomaly detected. Score: ' + data.anomaly_score);
        }
        document.getElementById('connection-status').innerHTML =
            'Connected — Last update: ' + time;
    } catch (e) {
        document.getElementById('connection-status').textContent =
            'Connection lost — Attempting to reconnect';
    }
}
setInterval(fetchData, 2000);
fetchData();
/* ── History page functions ── */
let currentPage = 1;
async function loadHistory(page = 1) {
    const from    = document.getElementById('filter-date-from')?.value || '';
    const to      = document.getElementById('filter-date-to')?.value   || '';
    const anomaly = document.getElementById('filter-anomaly')?.value   || '';
    const alert   = document.getElementById('filter-alert')?.value     || '';
    const url = `/api/history?page=${page}&from=${from}&to=${to}`
              + `&anomaly=${anomaly}&alert=${alert}`;
    const res  = await fetch(url);
    const data = await res.json();
```

```
    const tbody = document.getElementById('readings-body');
    if (!tbody) return;
    tbody.innerHTML = '';
    data.readings.forEach(r => {
        const anomalyBadge = r.is_anomaly
            ? '<span class="badge badge-anomaly">Anomaly</span>'
            : '<span class="badge badge-normal">Normal</span>';
        const alertBadge =
            `<span class="badge
badge-${r.alert_level}">${r.alert_level}</span>`;
        tbody.innerHTML +=
            `<tr>
              <td>${r.id}</td>
              <td>${new Date(r.timestamp).toLocaleString()}</td>
              <td>${r.vibration}</td>
              <td>${r.strain}</td>
              <td>${r.temperature}°C</td>
              <td>${anomalyBadge}</td>
              <td>${alertBadge}</td>
              <td>${r.anomaly_score.toFixed(3)}</td>
            </tr>`;
    });
    renderPagination(data.total_pages, page);
}
function renderPagination(total, current) {
    const div = document.getElementById('pagination');
    if (!div) return;
    div.innerHTML = '';
    for (let i = 1; i <= total; i++) {
        const btn = document.createElement('button');
        btn.className = 'btn btn-sm ' +
            (i === current ? 'btn-primary' : 'btn-outline-secondary');
        btn.textContent = i;
        btn.onclick = () => { currentPage = i; loadHistory(i); };
```

```
        div.appendChild(btn);
    }
    if (total > 1) {
        const next = document.createElement('button');
        next.className = 'btn btn-sm btn-outline-secondary';
        next.textContent = 'Next';
        next.onclick = () => {
            if (currentPage < total) { currentPage++;
loadHistory(currentPage); }
        };
        div.appendChild(next);
    }
}
function applyFilters() { loadHistory(1); }
async function exportCSV() {
    const from    = document.getElementById('filter-date-from')?.value || '';
    const to      = document.getElementById('filter-date-to')?.value   || '';
    const anomaly = document.getElementById('filter-anomaly')?.value   || '';
    const alert   = document.getElementById('filter-alert')?.value     || '';
    window.location.href =
        `/api/export?from=${from}&to=${to}&anomaly=${anomaly}&alert=${alert}`;
}
if (document.getElementById('readings-body')) loadHistory(1);
```

APPENDIX — RELEVANCE TO PROGRAMME OUTCOMES (POs) AND SUSTAINABLE DEVELOPMENT GOALS (SDGs)

A. Mapping to Programme Outcomes (POs)

The skills and competencies that the graduates of the B.Tech Artificial Intelligence and Data Science programme are supposed to exhibit are known as Programme Outcomes (POs). The table that follows maps the major aspects of this project to the pertinent POs as stipulated by the NBA accreditation framework.

Table A.1: Mapping of Project to Programme Outcomes

PO No.	Programme Outcome	How This Project Addresses It
PO1	Engineering Knowledge — Apply knowledge of mathematics, science, and engineering fundamentals to solve complex problems	The project applies mathematical foundations of machine learning including Isolation Forest decision boundaries, StandardScaler normalization, and statistical anomaly scoring through the decision_function method. Signal processing concepts from structural engineering underpin the sensor simulation logic.
PO2	Problem Analysis — Identify, formulate, and analyze complex engineering problems	The project begins by analyzing the specific gap in existing SHM systems — the absence of a practical, deployable software layer — and formulates a solution architecture that directly addresses the identified barriers of cost, hardware dependency, and lack of real-time intelligence.
PO3	Design and Development — Design solutions for complex engineering problems	The three-layer modular architecture (Sensor & Data Ingestion, Processing & Storage, Client & Alert) represents a deliberate engineering design that balances functionality, extensibility, and deployability. Each design decision — REST API format, ORM usage, dual-layer detection — was made to solve a specific identified problem.
PO4	Conduct Investigations — Use research-based knowledge to investigate complex problems	The literature review in Chapter 2 systematically surveys five major works in SHM research and identifies their specific merits and limitations. These findings directly informed the design decisions made in this project, particularly the

		choice of unsupervised ML algorithms and the software-first approach.
PO5	Modern Tool Usage — Use modern engineering tools, techniques, and resources	The project uses Python 3.10, Flask, Scikit-learn, SQLAlchemy, Chart.js, Twilio, Flask-Mail, and SQLite — all current industry-standard tools. The REST API design follows modern IoT integration standards compatible with ESP32 and Arduino microcontrollers.
PO6	Engineer and Society — Apply reasoning to assess societal, health, safety, and environmental impacts	The project directly addresses a societal safety problem — the risk of undetected structural deterioration in public infrastructure. The platform is designed to give engineers earlier warning of structural anomalies, directly reducing the risk of catastrophic infrastructure failure and its human and economic consequences.
PO9	Individual and Team Work — Function effectively as an individual and as a member of a team	This project was carried out collaboratively by three team members, with responsibilities divided across modules — sensor simulation and ingestion, backend processing and ML, and dashboard visualization and alerting — while maintaining architectural consistency across all components.
PO10	Communication — Communicate effectively through reports, documentation, and presentations	The project is documented through a comprehensive technical report covering problem definition, literature review, system design, implementation, results, and testing. The system itself includes a user-facing dashboard designed for clear communication of structural health status to non-technical stakeholders.
PO11	Project Management — Demonstrate knowledge and understanding of engineering management principles	The project was planned and executed in a modular sequence — software architecture first, simulation layer second, ML integration third, dashboard fourth — reflecting systematic project management and milestone-based development.

PO12	Life-long Learning — Recognize the need for and engage in independent and life-long learning	The project incorporates automated ML model retraining every 24 hours, reflecting the understanding that models must adapt continuously as real-world conditions change. The architecture is designed to support future upgrades — LSTM models, cloud deployment, real IoT sensors — without requiring a system redesign.

B. Mapping to Programme Specific Outcomes (PSOs)

There are other competencies in the AI and Data Sciences field, Programme Specific Outcomes.

Table A.2: Mapping of Project to Programme Specific Outcomes

PSO No.	Programme Specific Outcome	How This Project Addresses It
PSO1	Apply Artificial Intelligence and Data Science techniques to solve real-world problems	The project applies unsupervised machine learning — Isolation Forest and One-Class SVM — to solve the real-world problem of structural anomaly detection without requiring labeled failure data, which is rarely available in infrastructure monitoring contexts.
PSO2	Design and develop intelligent systems using modern AI tools and frameworks	The system is an end-to-end intelligent monitoring platform built using Python, Scikit-learn, Flask, and SQLAlchemy. The dual-layer detection architecture combines rule-based and ML-based intelligence in a production-ready software framework.
PSO3	Analyze and visualize data to extract meaningful insights for decision-making	The dashboard presents live sensor readings, anomaly scores, trend graphs, and historical filtered views — all designed to help engineers extract actionable insights from continuous sensor data streams and make informed maintenance decisions.

C. Mapping to Sustainable Development Goals (SDGs)

The United Nations Sustainable Development Goals (SDGs) are the universal system of 17 objectives that should be used to create the better and more sustainable future. The project is directly related to the following SDGs.

SDG 9 — Industry, Innovation and Infrastructure

Target 9.1: To achieve sustainable and resilient high-quality infrastructure to facilitate economic growth and human-well-being.

Target 9.4: Improve infrastructure and refurbish sectors to make them sustainable, ensuring that their efficiency in using resources is higher and that they are more reliant on clean and environmentally-friendly technologies.

The contribution made by this project:

It is a direct project that enables SDG 9 because the software platform will allow continuous and automated monitoring of civil infrastructure - bridges, buildings, dams, and industrial facilities. The system fills the gap between costly hardware-intensive SHM systems and the realistic requirements of smaller cities and infrastructure operators with limited resources. The platform allows more sustainable infrastructure monitoring practices to be established by enabling the expansion of structural health monitoring to be made accessible without bringing in any substantial hardware expenses. The IoT-enabled architecture fits in SDG 9.4, as it allows retrofitting the existing buildings with intelligent monitoring functionality at a low cost.

SDG 11 — Sustainable Cities and Communities

Target 11.1: It will provide access to safe infrastructure to everyone.

Target 11.B: Develop more cities that implement policies and plans that are integrated to achieve inclusion, resource efficiency, and disaster resilience.

The contribution made by this project:

Goal 11 demands safe, resilient, and sustainable cities and communities. A collapse of infrastructures, such as bridge collapses, building failures, breaking of dams, and others, are among the worst dangers to urban safety. The project is relevant to SDG 11 as it offers a tool that allows cities to keep track of their structural assets, identify any deterioration at an early stage, and act in time before failure. The low cost modular software platform is specifically tailored to be implemented by smaller urban governments that cannot afford full hardware based SHM systems and structural safety monitoring has become more accessible to the urban administrations operating at various degrees of urban development.

SDG 13 — Climate Action

Target 13.1: Build resilience and adaptive capacity to climatic risks.

The contribution made by this project:

Climate change is hastening the worsening of infrastructure by means of rising temperature extremes, flooding and unpredictable load events. This temperature monitoring aspect of this system is directly applicable in the process of monitoring structural stress caused by temperatures, which is an increasing cause of concern as temperatures increase. The fact that the system is capable of tracking changes in the temperature-induced vibration and indicating anomalies adds to the resilience of infrastructure to the stress of climate. The daily sinusoidal simulation of temperature applied during development is a specific simulation that embraces the thermal expansion impacts that are being become more and more evident due to climate change.

SDG 17 — Partnerships for the Goals

Target 17.6: Uplift science, technology, and innovation sharing and collaboration.

The contribution made by this project:

The article on this project has been posted to the ASEAN Journal of Scientific and Technological Reports (AJSTR) to make contributions to the world of research. The open, modular software architecture should be extensible and shareable to allow other researchers and practitioners to build on it. The software-first strategy that is exhibited in this project offers a model that can be duplicated by other institutions and modified to suit their infrastructure monitoring requirements.

D. Summary Table — SDG Contribution

Table A.3: Summary of SDG Contributions

SDG	Goal Title	Contribution Level	Key Connection
SDG 9	Industry, Innovation and Infrastructure	Direct	Low-cost software SHM platform for resilient infrastructure
SDG 11	Sustainable Cities and Communities	Direct	Early warning system for urban infrastructure safety

SDG 13	Climate Action	Indirect	Temperature-induced stress monitoring for climate resilience
SDG 17	Partnerships for the Goals	Indirect	Published research and open modular architecture for knowledge sharing

Software-First Structural Health Monitoring (SHM) platform for Real Time Anomaly Detection

1st Dr. S. Chakaravarathi
Professor
Department of AI&DS
Panimalar Engineering College
chakaravarathi2603@gmail.com

2nd P. R. Gopikashree
Student
Department of AI&DS
Panimalar Engineering College
gopikassakipog@gmail.com

3rd M. Devadarshini
Student
Department of AI&DS
Panimalar Engineering College
vasudevakrishnan171102@gmail.com

4th S. Dhivyadharshini
Student
Department of AI&DS
Panimalar Engineering College
dd07astro@gmail.com

Abstract*—The failure of infrastructure is not something to make newspapers but pocket lives and billions in clean up. Bridges fracture, buildings tilt, dams acquire stress fractures served out through years of weight. The point is that the majority of these problems are formed over a long period and remain unnoticed until the moment when it is already too late. Manual inspections are useful, but they are not routine, are labor intensive and in practice cannot be expected to find everything. The paper presents a software based Structural Health Monitoring (SHM) system which we have developed to address this problem in a different perspective. We did not begin with costly hardware but rather got the software architecture correct initially. Simulated sensor data on vibration, strain and temperature are processed in a continuous manner, and a dual-layer anomaly detector system consists of threshold-based rules to flag anomalies immediately and unsupervised machine learning with Isolation Forest and One-Class SVM models to recognize patterns in greater depth. Beneath the hood, it is powered by a Flask back-end, which is supported with SQLite and SQLAlchemy to store data and feed into a web dashboard where users may view live sensor data, historical trends, and automatically alerted through email and SMS. The ML models are trained using readings of the sensors in history and they automatically retrain after every 24 hours to keep up with the times. Preliminary testing with simulated streams was promising: each of the two detection methods performed well, alerts had been emitted when needed, and the export and visualization capabilities could be scaled up and down. The final product is a platform that is deploy-able, can be extended and really useful to providing engineers on the ground.

***Keywords*—Structural Health Monitoring (SHM), IoT, Anomaly Detection, Isolation Forest, One-Class SVM, Machine Learning, Flask, Real-Time Dashboard, Smart Infrastructure**

I. Introduction

Structural Health Monitoring has been brought into the background as one of the increasingly significant engineering disciplines of the past few decades and to be honest it is not hard to understand why. The bridges, tunnels and dams that are relied upon daily by billions of people were not constructed to last forever. They grow old, they take on stress and as time progresses those materials within them transform in some ways that may not necessarily be observed on the surface. Whenever something does go amiss, the after-effects are usually disastrous structural collapses that have claimed human lives, entire transit networks, and instigated recovery measures worth billions of dollars. That is why it becomes quite obvious that there is the case of the necessity to monitor these structures continuously and automatically.

The conventional method of regular manual inspections of the system with integrated sensor networks has performed fairly well, yet it can be subjected to actual limitations. The frequency by which a structure can be inspected is limited and much can occur in between inspections. The sensor-based systems developed over the years are also prone to high initial investment in hardware, calibration equipment, data acquisition equipment, which makes them unaffordable to most smaller projects or resource-limited teams. These are not insoluble issues but they have prevented further usage of the field.

The entry of the IoT, cloud computing, wireless networks, and machine learning is what has changed the picture significantly. It is possible with all these technologies to create monitoring systems that not only gather raw data but can also analyze it, give it a visual representation and take action to resolve issues in real time. This change has been noticed by the research fraternity. There has been a boom of research on wireless sensor nodes to monitor vibrations, deep learning to identify cracks, and hybrid systems that integrate analytics on a cloud with IoT infrastructure in the last few years.[6][23][26]

With that said, there is the gap that is not discussed so extensively. The majority of this work puts emphasis on either hardware performance or on algorithm sophistication. The software aspect of how data moves around the system, how it is stored and accessed, how the ML models are created and revised, how all this is available to the people who actually have to do something with it, is often considered secondary. This has the consequence of making most research prototypes challenging to implement in practice, and the transition between a laboratory demonstration and a practical field system even more challenging than it ought to be.

The present project is an effort to fill that gap. Instead of beginning with sensors, we were initially concerned with developing a clean and well-organized software platform. It uses a Flask back-end to run simulated vibration, strain, and temperature records and executes them using both a set of rule-based threshold checks and trained ML models namely

Isolation Forest and One-Class SVM and stores all of it with the help of SQLAlchemy and SQLite.[14] A web-based dashboard provides real time readings, ML anomaly scores and maintains a searchable history. The design is designed as a modular system with the purposeful addition of real IoT sensors, the database upgrade, or the change of ML models are all potentially possible with the help of updating the basic system.

II. RELATED WORKS

SHM is a long enough research that now has a rather well-established literature to use. The majority of early projects focused on sensor networks accelerator, strain sensors, and thermistors built into and out of structures themselves, with the data sent wirelessly to a central system. Such constructions are fair when well maintained and installed. The issue is that sensor calibration changes with time, batteries are exhausted, hardware is broken by the very environment that it is being sensitized on and the initial cost of instrumenting a large structure can be high.

The machine learning wave struck SHM research very badly, and it is quite understandable. In the case that you have continuous sensor streams off a structure, it appears there is an obvious use case of training a model to understand what normal looks like and indicating deviations. Scholars have experimented with simple classification algorithms all the way to the deep neural networks, and some of the results published appear to be very good. The snag though not always given enough attention in the papers is that these models require labeled training data. In actual infrastructure monitoring, the examples of anomalies are uncommon, imprecise and difficult to label with certainty. This is exactly the reason why unsupervised methods, such as Isolation Forest and One-Class SVM, have become popular as learners of normal behavior do not have to be shown examples of faults indicated with labels.[8][12]

The logic of Isolation Forest is to divide the feature space at random and watch the rate of isolation of single readings. The abnormal readings that appear to be well away on the dense normal cluster are separated in fewer splits and are given lower anomaly scores. One-Class SVM is the opposite, and it fits a boundary of normal distribution of data in a high dimensional feature space with the use of the RBF kernel. Any reading beyond this limit is identified as an anomaly. Both are fairly appropriate to situations in SHM where there is plenty of normal operating data, but very few fault examples.

More recently, this has changed somewhat to the focus on data infrastructure in how you actually store, query, and visualize the streams of data continuous monitoring produces. Cloud systems and edge computing are both considered approaches to managing the challenge of distributing the computational load without necessarily having everything on-site.[28] A topic that has always received little service in the literature is the dashboard interface which engineers and safety managers actually operate. The overwhelming majority of SHM articles contain a time-series plot or two demonstrating that the system is actually operational, yet fewer of them consider the visualization layer to be something to be constructed well.[16][20]

When put collectively, the literature indicates a uniform gap. Hardware and ML have certainly been done in a sophisticated way, but the software scaffolding to turn a monitoring system into something that is deployable and not just a prototype is relatively underserved. That is the area that this project is targeted.

III. PROPOSED METHODOLOGY

The system constructed is not based on the conventional SHM research template. Majority of work in this space is initiated by sensors or algorithms and expands out. It was the reverse of what we did and designed the software architecture first and then designed everything around it. The rationale is simple where in the event that the underlying framework is stable and modular, the addition of additional hardware or more advanced analytics in the future becomes a question of plugging in instead of being careful to re-create anew.

At the most basic level, the system has 3 layers: data ingestion, processing and storage and the client/alert interface illustrated in Fig. 1. Every layer has a well defined responsibility and modifications in any layer do not affect the rest in an unpredictable way. The ML models exist in the processing layer with the threshold logic, hence changing one model with another or even adding a new one does not need to touch the API layer or the database schema.

Currently, the system is controlled completely by the simulated sensor data vibration and strain and temperature data produced by Python scripts that simulate the sort of values you would observe of a real structure in both normal and abnormal conditions. That's intentional. The correct first thing to do before subjecting the messiness of real hardware to the software architecture is to validate the software architecture with clean controllable data. Physical sensors are added when needed, either by using Arduino boards or ESP32 modules, or by using dedicated IoT devices connected to MQTT or HTTP the ingestion layer does not need changes in the backend to accept them.

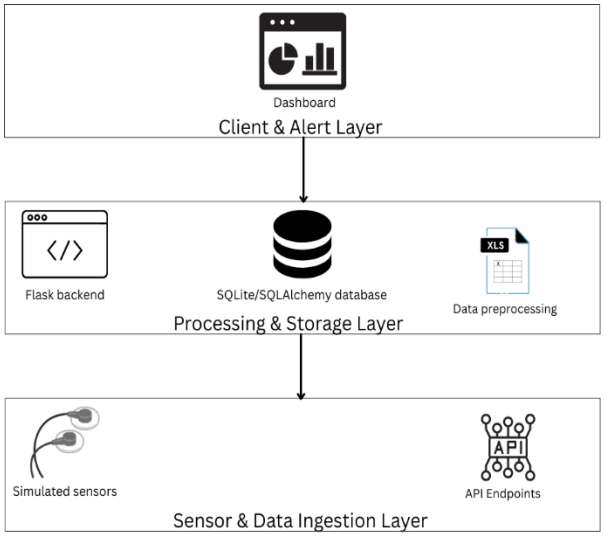

Fig. 1. Proposed SHM system System Architecture.

A. Sensor & Data Ingestion Layer

All monitoring systems require some data to operate with and because the objectives of this project were to test the software architecture and not test certain hardware, we

decided to use some simulated inputs. Python scripts create readings of vibration, strain, and temperature which remain within realistic values in most situations, with spikes being made occasionally to provide the anomaly detection logic something to detect. Temperature is a daily trend that includes thermal noise. Vibration is associated with temperature to recreate the effects of thermal expansion. The variation of strain is realistic randomness.

Information is transferred to the ingestion layer to the backend via regular API calls in the standard REST API and over the HTTP protocol. This was a conscious decision HTTP is what is basically spoken by every IoT microcontroller in nativity, that is, when the actual hardware is eventually attached it is able to start transmitting data to the identical endpoints instantly. No backend rewrites needed. The ingestion layer does not pay attention to the origin of its data, simulated scripts and physical sensors appear the same as the rest of the system after it is ingested into the pipeline.

B. Processing & Storage Layer

The back-end of the Flask is where the hard work actually occurs. On a reading arrival, it is initially validated and preprocessed to check the presence or absence of values and bad values, normalizes the values to a common range with StandardScaler, and stamps all the values with a uniform time measure. The entry point of clean data is also necessary due to the fact that inconsistent data will silently corrupt downstream ML logic in a way that is difficult to debug later.

After preprocessing, every reading is processed on two parallel abnormality detection programs. The former is threshold checking at a price, where the vibration will be a warning at 2.0 and critical at 2.5, strain will be a warning at 0.5 and critical at 0.7 and temperature will be a warning at 35C and critical at 40C. This provides instant and transparent classification, which is easily explained and optimized. The latter route involves the trained ML models. The Isolation Forest model has 100 estimators and contamination parameters of 0.1 indicating that it anticipates about 10 percent of the readings to be anomalous. One-Class SVM has automatic gamma scaling and RBF kernel. Both models take five input characteristics namely vibration, strain, temperature, hour of day and day of week the temporal characteristics allow the models to explain the natural variations in daily and weekly variations.

Table I presents the configuration settings and parameter values used in the proposed SHM system implementation.

S.No	Parameter Name	Explanation	Assigned Value
1	ML model — primary	Anomaly detection algorithm used as the main detection model	Isolation Forest
2	ML model — secondary	Comparative unsupervised model used alongside the primary	One-Class SVM
3	Number of estimators	Total decision trees built in the Isolation Forest ensemble	100
4	Contamination factor	Expected proportion of anomalous readings in training data	0.1 (10%)
5	SVM kernel	Kernel function used by One-Class SVM to define decision boundary	RBF
6	Gamma (SVM)	Kernel coefficient controlling the influence of training samples	Scale (auto)
7	Feature count	Number of input features fed to both ML models per reading	5
8	Input features	Sensor and temporal values used as model inputs	Vibration, strain, temperature, hour of day, day of week
9	Feature scaling	Normalization method applied before model training and prediction	StandardScaler
10	Training data window	Historical period from which training samples are collected	Last 7 days
11	Minimum training samples	Minimum readings required before a model can be trained	50
12	Model retraining interval	Time after which the active model is automatically retrained	24 hours
13	Vibration threshold — warning	Vibration value above which a warning alert is triggered	2.0
14	Vibration threshold — critical	Vibration value above which a critical alert is triggered	2.5
15	Strain threshold — warning	Strain value above which a warning alert is triggered	0.5
16	Strain threshold — critical	Strain value above which a	0.7

S.No	Parameter Name	Explanation	Assigned Value
		critical alert is triggered	
17	Temperature threshold — warning	Temperature (°C) above which a warning alert is triggered	35°C
18	Temperature threshold — critical	Temperature (°C) above which a critical alert is triggered	40°C
19	Alert fatigue limit	Maximum alerts of the same severity allowed per hour	5
20	Dashboard refresh rate	Frequency at which the live dashboard polls for new readings	Every 2 seconds

Table I. Configuration settings and parameter values of the proposed SHM system.

The ML models will be trained using the previous 7 days of readings, whereby a minimum of 50 samples is required before the models can start training. The metadata training time, sample size, parameters are stored in the database as a model. Background check is done after every 24 hours and automatically retrains the active model in case it is older than the retraining period and maintains the anomaly detector up to date as the structural behavior patterns change with time.

Table II is a thorough comparison table of both ML models used in this system, which shows the difference in the algorithm, but more importantly the settings used in the proposed implementation.

Parameter	Isolation Forest	One-Class SVM
Algorithm type	Tree-based ensemble	Kernel-based boundary
Core principle	Isolates anomalies via random splits; anomalous points are isolated in fewer steps	Learns a tight boundary around normal data in high-dimensional space
Training speed	Fast	Slower on large datasets
Memory usage	Low	Moderate to high
Works without labeled data	Yes	Yes
Contamination / outlier control	Explicit — `contamination` parameter	Implicit — `nu` parameter
Sensitivity to noisy training data	Low	High
Interpretability	Moderate (anomaly score via path length)	Low (kernel decision boundary)
Input features used	Vibration, strain, temperature, hour of day, day of week	Vibration, strain, temperature, hour of day, day of week
Configuration in proposed system	n_estimators = 100, contamination = 0.1, random_state = 42	kernel = RBF, gamma = scale
Role in proposed system	Primary anomaly detection model	Secondary / comparative model

Table II. Comparison of ML models used in the proposed SHM system.

All is written into SQLite using the ORM of SQLalchemy. With an ORM, the database logic is not tied to the rest of the application, and thus, it is easy to switch to PostgreSQL, MySQL, or Firebase in the future without changing a significant amount of code. The stored recordings contain crude sensor data, time, ML anomaly flags, anomaly scores and alert levels. Queries are maintained fast by three database indexes on the basis of timestamps, level of alert and flag of anomaly as data continues to build.

C. Client & Alert Layer

The aspect of the system which most individuals will actually be using is the dashboard and thus we took actual care in making it truly useful as opposed to simply being technical. It shows real-time graphs of all three sensor types vibration, strain, and temperature and has a summary table with the latest readings and their ML anomaly scores and alert status as well as simple statistical summaries that provide a rough idea of the behavior of the structure in the past 24 hours.

The dashboard sets the flag of an anomaly as soon as the backend notices it, in a color coded format based on its severity. In remote monitoring cases where no one is staring at the screen throughout the day, the architecture enables the extension of both the alerts to SMS through Twilio and the email through Flask-Mail. There is an alert fatigue protection system that ensures that the number of alerts of similar severity level is sent on not more than five occasions within the same hour, a feature that is applicable in practice and ensures that engineers are not overwhelmed by numerous notifications during a prolonged event of anomaly.

The filtering options allow users to retrieve historical data on any given time or type of anomaly and filter can be exported as CSV to further examine them in other applications. The database also stores the alert history and thus you can look into the history of when an alert was sent, what activated it and how successful it was sent.

IV. EXPERIMENTS AND RESULT

Construction of the system was much concerned with actualizing the architecture choices of the preceding section. Every aspect was required not only to appear well on a diagram, but to actually cooperate with others. That is how each of the pieces got assembled.

A. Data Simulation & Ingestion

As the objective of this project was to make the software work properly, first and then we would worry about the hardware, we coded Python scripts to simulate physical sensors. In sensorservice.py, sensor simulation produces temperature under daily sinusoid with the center temperature of 25C and random noise of 5C. The temperature is derived

into vibration with a base of 0.5 plus a thermal expansion factor, and random noise of 1.5 is tailored to a minimum of 0.1. There is uniform strain with 0.0 to 0.8 as an independent random variable.

Each reading is packaged in the form of a JSON object with the sensor value, timestamp, sensor type, anomaly status and alert level. The selection of this payload structure is due to the fact that it is similar to the default data that most IoT microcontrollers transmit. An Arduino, or ESP32, eventually replaces the Python, just sending the same JSON to the same endpoint - the backend is unaware of the difference.

B. Backend Processing using Flask

Flask takes care of all the routing and processing logic in a structure based on Blueprint with blueprints that define main routes, API routes and history routes. All incoming sensor measurements pass through a common pipeline: the sensor service produces and stores the measurement, the ML service executes anomaly prediction and threshold check, assigns the alert level and the alert service sends notifications where necessary.

This anomaly detection in mlservice.py employs IsolationForest and OneClassSVM of scikit-learn. The two models employ StandardScaler to normalize features to train and predict. The predictanomaly approach tries to load the already trained model and auto-trains the new model in case there is no available model. Each reading is paired with the anomaly scores of the decision function and provides a continuous measure of the degree to which a particular reading is considered to be anomalous. The retraining needed model verifies the age at every call, and automatically initiates retraining when the model goes beyond the 24-hour threshold.

C. Database Management with SQLite/SQLAlchemy

The ORM of SQLAlchemy manages all the interactions with the database by providing the following three models: SensorReading: the reading of each sensor and its ML results, MLModel: the metadata of every trained model and AlertLog: the attempt of each alert and its result. The ORM-based implementation makes the system database-agnostic so that the swap of SQLite with PostgreSQL or MySQL should only change the connection string in the config. The SensorReading table has three composite indexes on it (on timestamp, alertlevel, and isanomaly) to ensure that queries remain fast in scale.

Sensors are recorded in the database with each reading and the time when it was taken, anomaly flag, ML anomaly score, and the alert level. It is this cumulative history that allows both trend analysis and retraining a model of prediction to take place the retrain pipeline simply queries this table of the last 1,000 readings in the past 7 days.

D. Dashboard & Visualization

The dashboard is developed using Flask templating engine and chart.js is used to render the chart. Vibration, strain, and temperature live data are shown in three separate panels which update after every 2 sec with new readings. The JavaScript frontend makes a call to the / sensor-data endpoint that activates the entire pipeline sensor generation, ML prediction, threshold check, alert dispatch and returns the result. Coupled with the live charts, a data table of the recent readings with their status of anomaly and their ML scores, and a statistics summary panel is present.

Historical filtering feature allows the user to choose time window and display the readings at that time period but filtered by sensor type or anomaly type. One can also export data as CSV to further analyze it in personal tools. The history page at / history gives a dedicated page to view the past sensor data and alert logs.

E. Alerting Mechanism

The alerting of the alerting system is performed through the alert service, which is the alerting system, alerting service.py, and which deals with email alerts using Flask-Mail and SMS alerts using Twilio REST API. The email notifications have a complete sensor reading history, ML anomaly score, and the violation of the threshold message. SMS notices are compacted to the size of characters. Both email and SMS are activated by critical alerts, whereas email is activated by alerts of warning level. The alert fatigue protection query screens the AlertLog table against alert notification of similar severity, and avoids the notification in case of five or more notification within the past hour. All the successful and unsuccessful alert attempts are recorded in the database with the recipient, content of the message, time and error details where necessary.

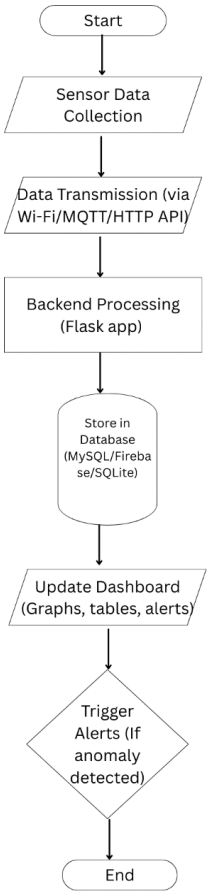

Fig. 2. Flow diagram of the proposed Structural Health Monitoring (SHM) system

Fig. 2 above depicts the flow diagram of how all this links end to end beginning with data entering the ingestion layer, through ML processing and storage, out to the dashboard and alert system. The modular design is based on the fact that

each stage is autonomous enough to be altered or replaced without the destruction of the rest.

Simulated data streams were used to test the functionality of every component of the system. These findings showed effective and consistent performance on all the modules.

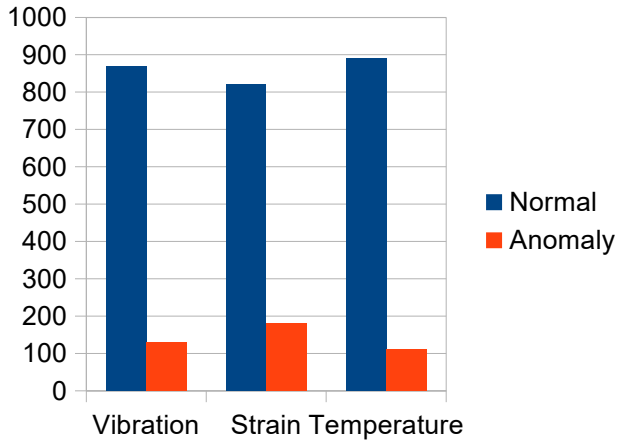

Fig. 3. Normal vs Anomaly sensor reading

Fig. 3 shows the statistics of the normal and anomalous values of all three types of sensors vibration, strain, and temperature according to the ML anomaly detection layer. In all the sensors, most readings were taken to be normal, which is in line with expectation of structural behavior in simulated conditions. Approximately 13 percent of vibration readings, 18 percent of strain readings and 11 percent of temperature readings were attributed to anomalous readings. The increased anomaly rate of strain is explained by the broader random variation of the simulated strain signal, which sometimes transverses the decision boundary of the trained model. This proves that the dual layer detection system is operating as planned and the ML layer is able to detect deviations and not over-flag normal operation.

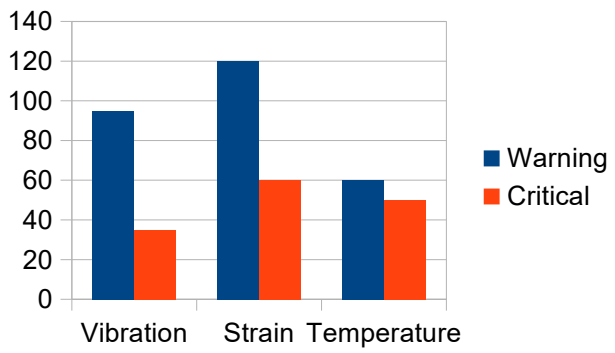

Fig. 4. Threshold breach count per sensor

Fig. 4 shows how many times each sensor passed the warning and critical limits set in the testing process. There were 95 warning breaches and 35 critical breaches. Strain registered 120 warning breaches and 60 critical breaches, which is more variable. The temperature registered 60 warning and 50 critical breaches averaging the simulated daily temperature cycle that intermittently touched down to the warning temperature of 35C. In every instance where threshold detection layer reacted instantly at the breach point, this substantiated the responsiveness of rule-based detection layer to real-time.

The results were good insofar as functionality was concerned. Vibration, strain and temperature sensor readings were processed and real time display was made, with no measurable latency, even with data frequency increased to levels that would be necessary in a real deployment. The threshold-based anomaly detector was used as desired and readings of the vibrations above the critical threshold were sounded immediately and the medium level of change of strain swung Warning-level alerts instead of shifting straight to Critical.

The ML anomaly detection layer was a helpful addition. Upon training the Isolation Forest model on enough data, the anomaly scores produced by this model were strongly correlated with the threshold-based system. This was also true in the case of One-Class SVM. Neither of the two models had issues with the five-feature input (vibration, strain, temperature, hour, weekday) and the auto-retraining process would start as desired when the model aged to 24-hour.

There was a good performance of database. The queries of individual several thousand historical records were still responsive in the dashboard and the CSV export and historical filtering capability were some of the more practically useful functionalities in testing. Fatigue protection (when alert) was also effective, but up to five alerts per hour when there were long periods of anomaly.

V. DISCUSSION

Although the results were promising, sincere shortcomings should be admitted. It is an experimental software architecture rather than a finished product and there are still areas that need to be developed before it can be used in the real world.

The ML layer has a cold-start drawback there must be at least 50 readings to train before it can start, that is, the ML detection layer will be inaccessible at first deployment. Both the contamination parameter of the Isolation Forest and the nu parameter of the One-Class SVM are set to 0.1 which in a production setting would have to be adjusted to the actual fault rates of the structure under observation. An individual contamination should not be representative of contamination in various types of structure or environments.

The system is also yet to be integrated with actual hardware. The ingestion layer was intentionally hardware-agnostic; however, integration problems do exist and there is a realism of timing issues, packet loss, sensor drift, or other protocol-related irregularities and inconsistencies which the simulated data cannot identify until a real deployment occurs.

SQLite can be used to develop and test a single structure but not to support high-concurrency or the multi-structure large scale deployments on the storage side. It would need to migrate to a postgreSQL or a cloud-native database first before the system could be utilized to monitor multiple large structures at the same time. This has been designed using the SQLAlchemy-based ORM to ensure that this migration is easily achieved although the actual migration is yet to be confirmed.

These weaknesses do not negate the basic results, although they do place the boundary between what has been demonstrated and what remains to be done before this can be transferred out of the research prototype and into a system capable of operating in the field.

VI. CONCLUSION

This study proves that software-first is a legitimate unexplored direction of Structural Health Monitoring studies. Most of the field is inclined towards hardware innovation or algorithm sophistication. Putting in the work to have the architecture correct modular layers, clean APIs, automated ML retraining, a usable dashboard interface, and the right data storage is so much better at the point of time of extension or scaling up of the system.

Other additions like the addition of real IoT sensors, production-grade database or even more advanced ML-models should be additions, and not a major rebuild. This is what this project was aimed to create and, judging by the results, it is doing so.

REFERENCES

[1] S. K. Laha, M. Pal, and S. Bhattacharyya, "Anomaly detection in SHM using spectral distance and t-SNE-GMM," Asian Journal of Civil Engineering,vol.25,pp.3711–3727,2024.

[2] T. J. Saravanan, M. Mishra, A. D. Aherwar et al., "Internet of Things (IoT)-based structural health monitoring of laboratory-scale civil engineering structures," Innovative Infrastructure Solutions, vol. 9, p. 110, 2024.

[3] J. Cho, K. J. Lim, J. Kim, Y. Shin, Y. S. Park, and J. Yeon, "Enhancing data quality management in structural health monitoring through irregular time-series data anomaly detection using IoT sensors," Buildings, vol. 14, no. 7, p. 2223, 2024.

[4] H. Zhang et al., "Structural health monitoring data analysis using deep learning techniques," in Proc. 8th International Conference on Electronic Information Technology and Computer Engineering (EITCE), 2024.

[5] M. Azimi, A. D. Eslamlou, and G. Pekcan, "Data-driven structural health monitoring and damage detection through deep learning: State-of-the-art review," Sensors, vol. 20, no. 10, p. 2778, 2020.

[6] L. Sun, Z. Shang, Y. Xia, S. Bhowmick, and S. Nagarajaiah, "Review of bridge structural health monitoring aided by big data and artificial intelligence," Journal of Structural Engineering, vol. 146, no. 5, 2020.

[7] D. Okur, "A novel approach for anomaly detection in vibration-based structural health monitoring using autoencoders in deep learning," Structural Control and Health Monitoring, 2025.

[8] K. Alissa et al., "Robust IoT security using Isolation Forest and One-Class SVM algorithms," Scientific Reports, 2025.

[9] Arvind Dagur; Karan Singh; Pawan Singh Mehra and Dhirendra Kumar Shukla. (2021). Artificial Intelligence based IoT for Smart Infrastructure Health Monitoring. Journal of Physics: Conference Series.

[10] A. Rajkumar, "Anomaly detection using unsupervised machine learning algorithms: a simulation study," Heliyon, vol. 10, no. 17, 2024.

[11] Ghosal, A. and A. Nandy. (2020). A Short Review on Different Clustering Techniques and Their Applications. Emerging Technology in Modelling and Graphics, pp. 69–83.

[12] S. Astuti and B. Purnomo, "Comparison of Isolation Forest and One-Class SVM in anomaly detection of gas pipeline operation," in Proc. 3rd International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA), IEEE, 2023.

[13] M. Malik, R. Rouf, K. Mazur, and A. Kontsos, "The industry internet of things (IIoT) as a methodology for autonomous diagnostics in aerospace structural health monitoring," Aerospace, vol. 7, no. 5, p. 64, 2020.

[14] Chen, B. and Z. Zhuang. (2020). Flask-based real-time IoT sensor data visualization for infrastructure monitoring systems. IEEE Access, 8: 112345–112358.

[15] A. Azad, S. Kim, Y. B. Cheon, and H. S. Kim, "Intelligent structural health monitoring of composite structures using machine learning, deep learning, and transfer learning: a review," Advanced Composite Materials, vol. 33, no. 2, pp. 162–188, 2024.

[16] Samudra, S.; Barbosh, M. and A. Sadhu. (2023). Machine Learning-Assisted Improved Anomaly Detection for Structural Health Monitoring. Sensors, 23(7): 3365. https://doi.org/10.3390/s23073365

[17] Mishra, M.; Lourenço, P. B. and G. V. Ramana. (2022). Structural Health Monitoring of Civil Engineering Structures by Using the Internet of Things: A Review. Journal of Building Engineering, 48: 103954.

[18] Flah, M.; Nunez, I.; Ben Chaabene, W. and M. L. Nehdi. (2021). Machine Learning Algorithms in Civil Structural Health Monitoring: A Systematic Review. Archives of Computational Methods in Engineering, 28(4): 2621–2643.

[19] Barros, J. and others. (2024). Comparison of Isolation Forest and One-Class SVM in Anomaly Detection of Gas Pipeline Operation. IEEE Conference Publication. IEEE Xplore.

[20] Nassif, A. B.; Talib, M. A.; Nasir, Q. and F. M. Dakalbab. (2021). Machine Learning for Anomaly Detection: A Systematic Review. IEEE Access, 9: 78658–78700.

[21] Toufigh, V. and I. Ranjbar. (2023). Unsupervised Deep Learning Framework for Ultrasonic-Based Distributed Damage Detection in Concrete: Integration of a Deep Auto-Encoder and Isolation Forest for Anomaly Detection. Structural Health Monitoring.

[22] Enhancing Critical Infrastructure Security: Unsupervised Learning Approaches for Anomaly Detection. (2024). International Journal of Computational Intelligence Systems, Springer Nature.

[23] Sofi, A.; Regita, J. J.; Rane, B. and H. H. Lau. (2022). Structural Health Monitoring Using Wireless Smart Sensor Network — An Overview. Mechanical Systems and Signal Processing, 163: 108113.

[24] Mishra, M. and P. B. Lourenço. (2024). Use of IoT for Structural Health Monitoring of Civil Engineering Structures: A State-of-the-Art Review. Urban Lifeline, Springer Nature.

[25] Bono, F. M. and others. (2024). Low-Cost, Low-Power Edge Computing System for Structural Health Monitoring in an IoT Framework. Sensors, 24(15): 5078.

[26] Wan, H. P.; Zhu, Y. K.; Luo, Y. and M. D. Todd. (2025). Unsupervised Deep Learning Approach for Structural Anomaly Detection Using Probabilistic Features. Structural Health Monitoring.

[27] M. T. Casas-Rodríguez et al., "AI in structural health monitoring for infrastructure maintenance and safety," Infrastructures, vol. 9, no. 12, p. 225, 2024.

[28] X. Wan, S. Guan, and Y. Tang, "Advancing bridge structural health monitoring: insights into knowledge-driven and data-driven approaches," Journal of Data Science and Intelligent Systems, vol. 2, no. 3, pp. 129–140, 2024.

[29] M. M. Azad et al., "Artificial intelligence and data analytics for structural health monitoring: a review of recent developments," Archives of Computational Methods in Engineering, 2025.

ANNEXURE 1		
STUDENTS PROJECT ROAD MAP		
NAME OF THE STUDENTS		REGISTER NUMBER
DEVADARSHINI M		211422243055
DHIVYADHARSHINI S		211422243067
GOPIKASHREE P R		211422243080
NAME OF THE SUPERVISOR: Dr. S. CHAKARAVARTHI, M.E., Ph.D		
DEPARTMENT: Artificial Intelligence and Data Science		
1	TITLE OF THE PROJECT	Software-First Structural Health Monitoring (SHM) Platform for Real-Time Anomaly Detection
2	RATIONALE (why the topic is important today in 3 sentences in bullet points)	• Structural failures in infrastructure such as bridges and buildings lead to severe economic loss and risk to human life. • Existing monitoring systems are expensive and not accessible for small-scale deployments. • A software-first SHM system enables affordable, scalable, and real-time monitoring solutions.
3	LITERATURE SURVEY (Top 5 articles utilized for finding the research gap and their SCOPUS impact factor)	• Farrar & Worden (2007) – SHM framework and vibration-based detection (High impact journal) • Lynch & Loh (2006) – Wireless sensor networks in SHM (SCOPUS indexed) • Li et al. (2015) – IoT-based SHM systems (High impact factor) • Ye et al. (2019) – Machine learning techniques in SHM (SCOPUS indexed) • Sun et al. (2020) – Cloud-based SHM platforms (High impact journal)

4	RESEARCH GAP (Maximum 3 sentences in bullet Points)	• Most SHM systems rely heavily on costly hardware and infrastructure. • Existing solutions lack real-time anomaly detection with integrated visualization. • Limited focus on software-first scalable architectures.
5	BRIDGING THE GAP (Maximum 4 sentences in bullet Points)	• Developed a software-first SHM system eliminating dependency on hardware. • Integrated dual-layer anomaly detection using threshold and ML techniques. • Built real-time dashboard for visualization and monitoring. • Designed system to be IoT-ready for future hardware integration.
6	NOVELTY (Maximum 3 sentences in bullet Points)	• Combines rule-based and ML-based anomaly detection in a single system. • Implements automatic ML model retraining for adaptive monitoring. • Introduces alert fatigue protection for real-world usability.
7	OBJECTIVES (Maximum 5 sentences in bullet Points)	• To design a software-first SHM monitoring system. • To simulate real-time sensor data for testing. • To implement anomaly detection using Isolation Forest and One-Class SVM. • To develop a real-time dashboard for visualization. • To create an alert system for early warning detection.
8	PROCESS METHODOLOGY (Maximum 7 sentences in bullet Points)	• Simulate sensor data (vibration, strain, temperature). • Send data via REST API to backend. • Validate and preprocess incoming data. • Apply normalization using StandardScaler. • Perform anomaly detection using threshold and ML models. • Store processed data in database.

		• Display results on dashboard and trigger alerts.
9	SIMULATION METHODOLOGY AND SIMULATION SOFTWARE REQUIREMENT (Maximum 4 sentences in bullet Points)	• Sensor data simulated using Python scripts. • Backend developed using Flask framework. • Machine learning implemented using Scikit-learn. • Visualization done using Chart.js and HTML/CSS.
10	DELIVERABLES & OUTCOMES (Maximum 4 sentences in bullet Points) (Technology,Prototype, Algorithm, Software, patent, publication, etc)	• Functional SHM web-based monitoring system. • Real-time anomaly detection algorithm implementation. • Interactive dashboard for visualization. • Scalable architecture ready for IoT integration.
11	PROJECT CONTRIBUTION IN REALTIME	Journal Paper
11	SUSTAINABLE DEVELOPMENT GOALS MAPPED WITH JUSTIFICATION	SDG 9 (Industry, Innovation, Infrastructure): Enhances infrastructure safety using technology
		SDG 11 (Sustainable Cities): Supports smart city monitoring systems
		SDG 12 (Responsible Consumption): Prevents infrastructure failure and wastage
12	PROGRAMME OUTCOME MAPPING (PO) WITH JUSTIFICATION	PO1: Applied AI/ML concepts for anomaly detection
		PO2: Designed real-time monitoring system
		PO3: Developed software solution for real-world problem
		PO4: Conducted analysis and validation of results

		PO5: Used modern tools like Flask, ML libraries, and visualization tools
13	Timeline	Milestones
	Review 1	During the first review phase, the project topic was finalized and the problem statement was clearly defined. A detailed literature survey was conducted to understand existing Structural Health Monitoring systems and identify research gaps. The initial project plan and objectives were also prepared and discussed with the supervisor.
	Review 2	In the second review phase, the system architecture and design were completed, including data flow diagrams and database design. The backend development using Flask and the implementation of machine learning models for anomaly detection were initiated. Initial testing of the system with simulated sensor data was also carried out.
	Review 3	During the final review phase, the complete system was implemented and integrated, including the dashboard visualization and alert mechanisms. The system was thoroughly tested and validated for performance and reliability. Finally, the project report and documentation were completed and prepared for submission.
SUPERVISOR SIGNATURE		